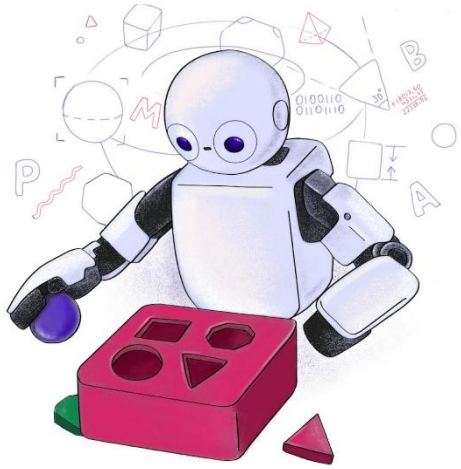
C24 – Inteligência Artificial:
Redes neurais recorrentes



Inatel

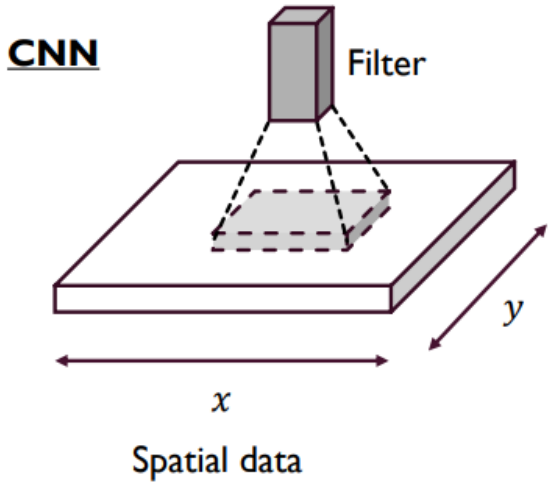
Felipe A. P. de Figueiredo
felipe.figueiredo@Inatel.br

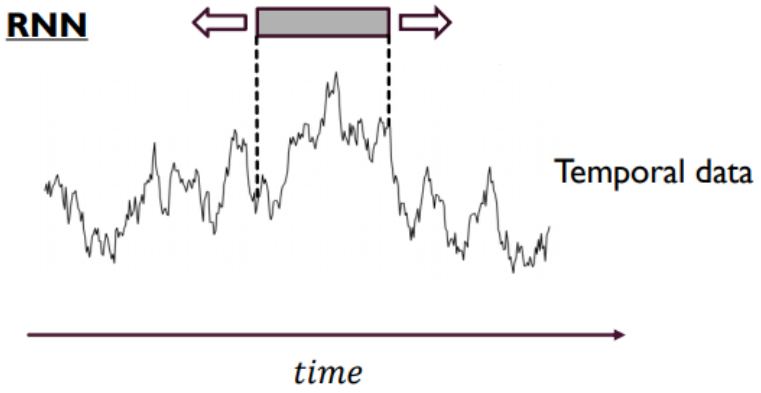
Motivação

- Muitos problemas envolvem **dados sequenciais**:
  - Voz
  - Texto
  - Séries temporais (e.g., EGC, EEG, preço de ações, desvanecimento de um canal)
  - Vídeo
- Para lidarmos com eles, precisamos **modelar dependências temporais!**
- Em outras palavras, existe **correlação temporal entre as amostras.**

Cada amostra tende a carregar informação correlacionada com as anteriores.

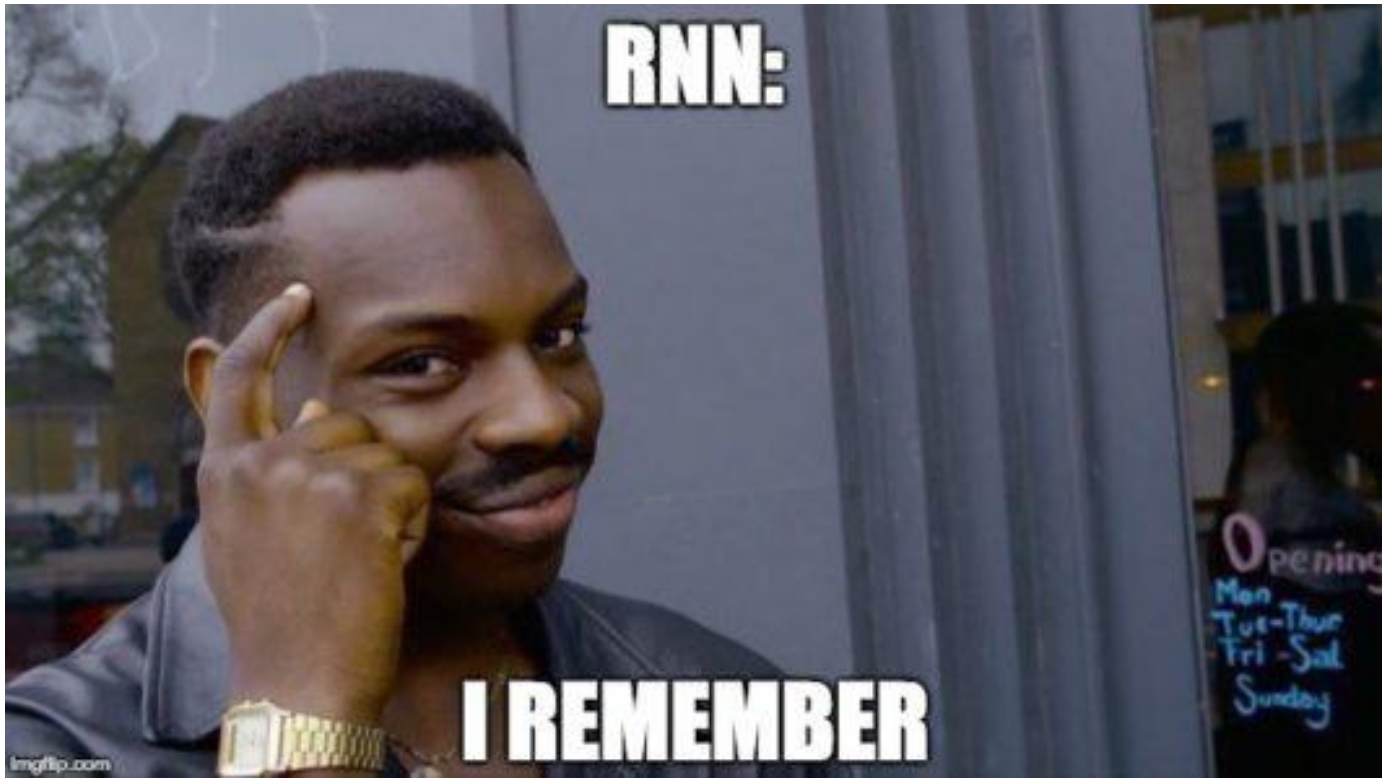
Limitação de redes tradicionais





- MLPs:
  - **Não armazenam informações de estados anteriores** (i.e., sem memória)
  - As **entradas** são tratadas de forma **independente**.
    - Isso faz com que elas **não capturem dependências temporais entre amostras**.
  - Por ser de alimentação direta, a saída **depende apenas da entrada atual**.
- CNNs:
  - Compartilham as limitações das MLPs, mas com uma exceção: elas **modelam relações espaciais**.
- Ambas famílias **exigem** que o **vetor de entrada** tenha um **tamanho predefinido e constante**.

Como podemos considerar as correlações temporais e trabalhar com entradas sem tamanho predefinido?

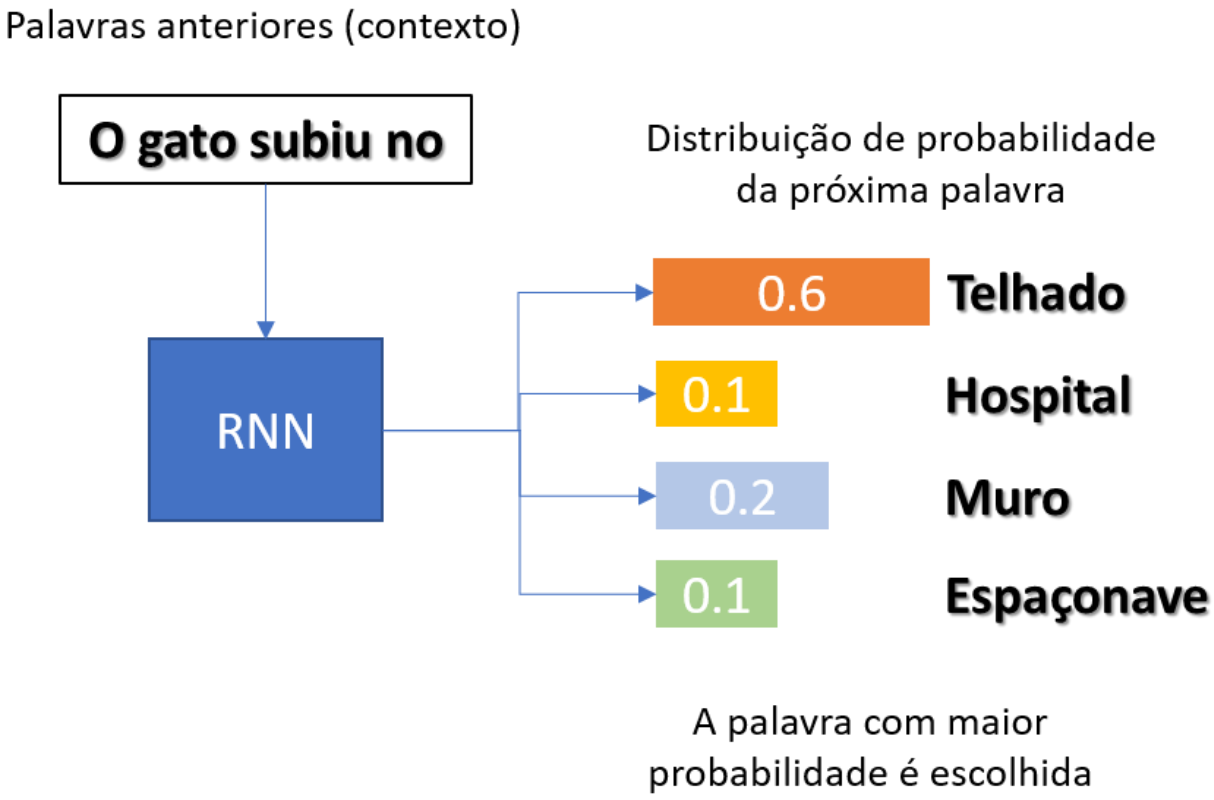


Redes neurais recorrentes (*recurrent neural networks* – RNNs) têm memória!

Aplicações de redes neurais recorrentes

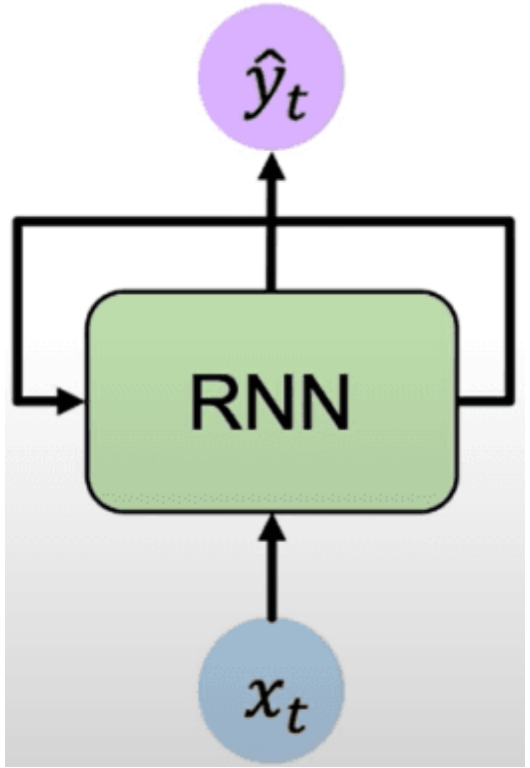
- Previsão de séries temporais (e.g., clima, ações, vendas, etc.)
- Tradução automática de texto
- Reconhecimento de fala
- Tradução fala para texto e vice versa
- Classificação de sentimentos
- Geração de texto
- Geração de descrições de texto para imagens (*image captioning*)
- Rastreamento de objetos

Padrões temporais



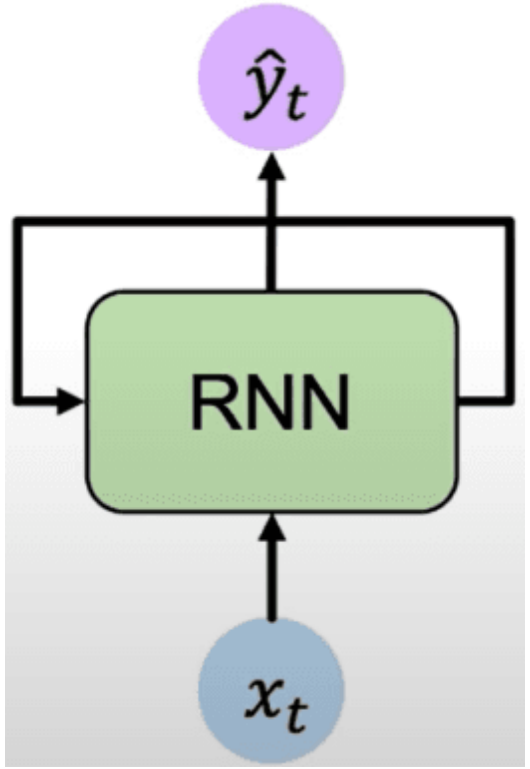
- CNNs aprendem padrões espaciais, enquanto RNNs aprendem padrões temporais.
- No exemplo ao lado, a RNN recebe um contexto, como o “**O gato subiu no**”, e apresenta as palavras candidatas com suas respectivas probabilidades estimadas.

Redes recorrentes



- Possuem **laços de realimentação**.
- Mantém um **estado interno** (**memória**) que **armazena informações** observadas até o momento.
- A **saída** da rede depende da **entrada atual** e do valor do **estado interno**.
- O **estado interno** mantém o histórico de **das entradas passadas**.

Redes recorrentes

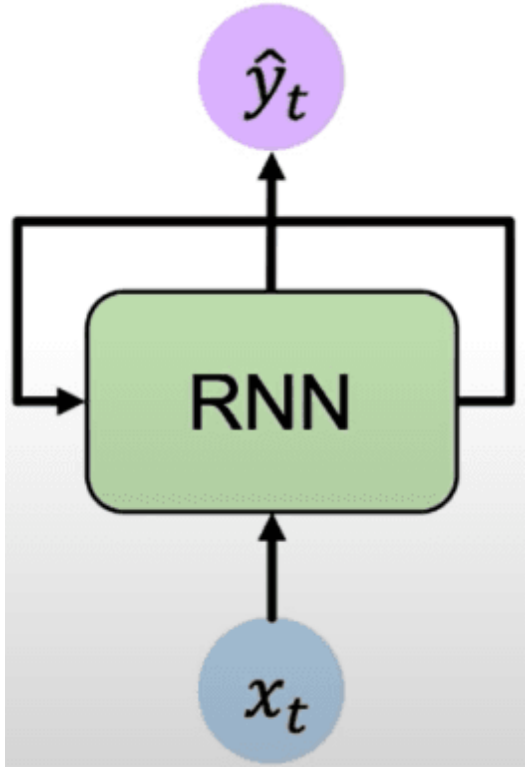


- Uma RNN utiliza **informações sequenciais**, **modelando as dependências temporais** entre as amostras de entrada.
- **Exemplo**: se quisermos **prever a próxima palavra** em uma frase, precisamos saber **quais palavras vieram antes dela**.
- Por exemplo, na frase:

  “O gato subiu no ____”

a rede usa as palavras anteriores para prever que “telhado”, “muro”, “sofá” têm maior probabilidade.

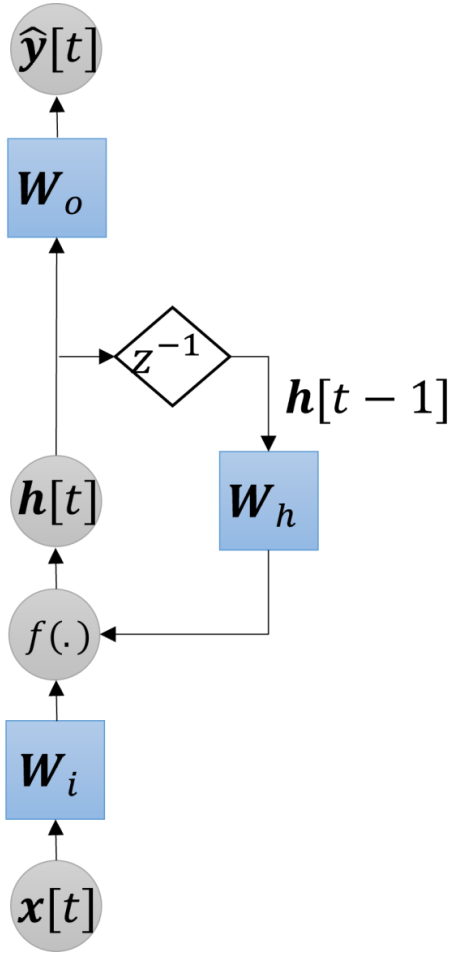
Redes recorrentes



- RNNs são projetadas para processar sequências de **comprimento variável**.
- Elas usam o **histórico de entradas anteriores** para **gerar uma saída**.
- Porém, como veremos, um problema das RNNs é que a **memória desaparece rapidamente** ao longo do tempo.

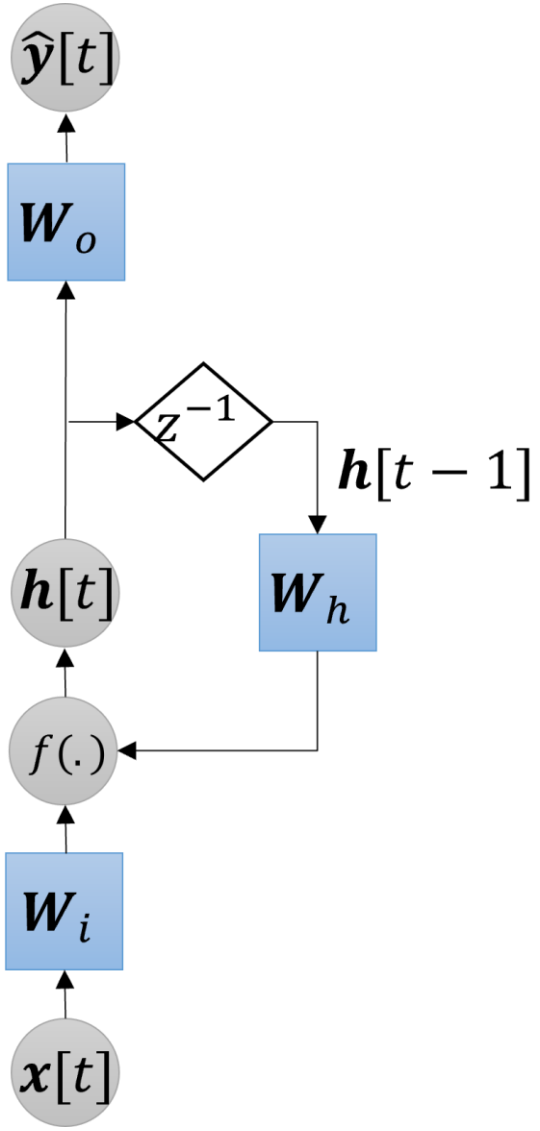
O que compõem as RNNs?

Neurônio (ou célula) recorrente



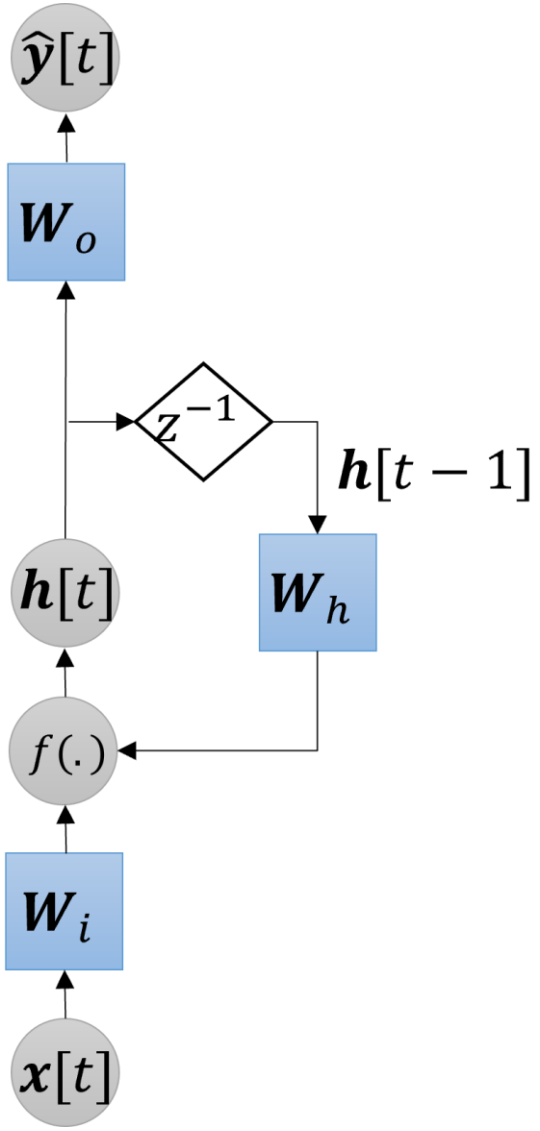
As RNNs são compostas por **neurônios recorrentes.**

Neurônio (ou célula) recorrente



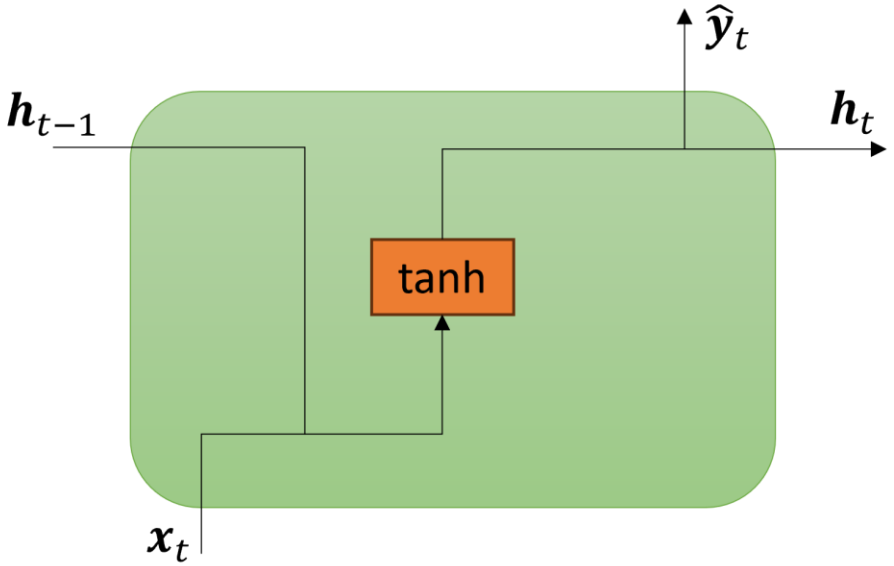
- $\boldsymbol{x}[t]$: entrada
- $\hat{\boldsymbol{y}}[t]$: saída
- $\boldsymbol{h}[t]$: estado interno ou oculto (**memória da rede**)
- $\boldsymbol{h}[t-1]$: estado interno anterior
- $\boldsymbol{W}_i$: pesos de entrada
- $\boldsymbol{W}_h$: pesos da camada recorrente (oculta)
- $\boldsymbol{W}_o$: pesos de saída
- z^{-1}: unidade de retardo de tempo
- $f(.)$: função de ativação

Neurônio (ou célula) recorrente



- As RNNs lidam com as dependências temporais entre amostras mantendo um **estado oculto**, $\boldsymbol{h}[t]$, que armazena informações de todos os passos de tempo anteriores da sequência (**memória**).
  - $\boldsymbol{h}[t]$ é uma representação compacta da história da sequência.
- O mecanismo de **recorrência** permite que a rede aplique o contexto aprendido a partir de entradas anteriores às etapas atuais e futuras.
- Uma célula recorrente **aprende padrões presentes** nas sequências.

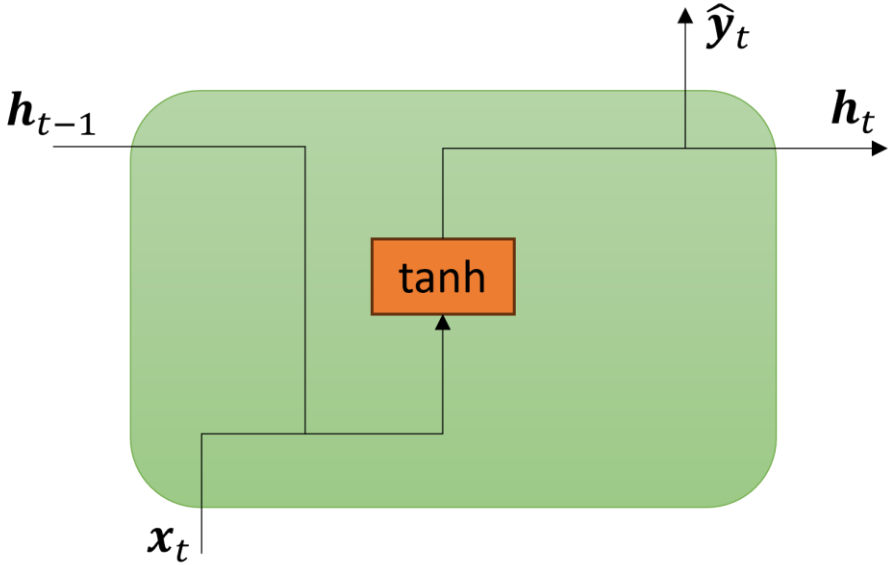
Neurônio recorrente



- Representação **simplificada** do neurônio.
- Ela omite a maioria das matrizes de pesos, mostrando uma caixa laranja que corresponde a uma camada densa com função de ativação do tipo tangente hiperbólica (tanh).
- Essa representação é bastante utilizada na literatura.

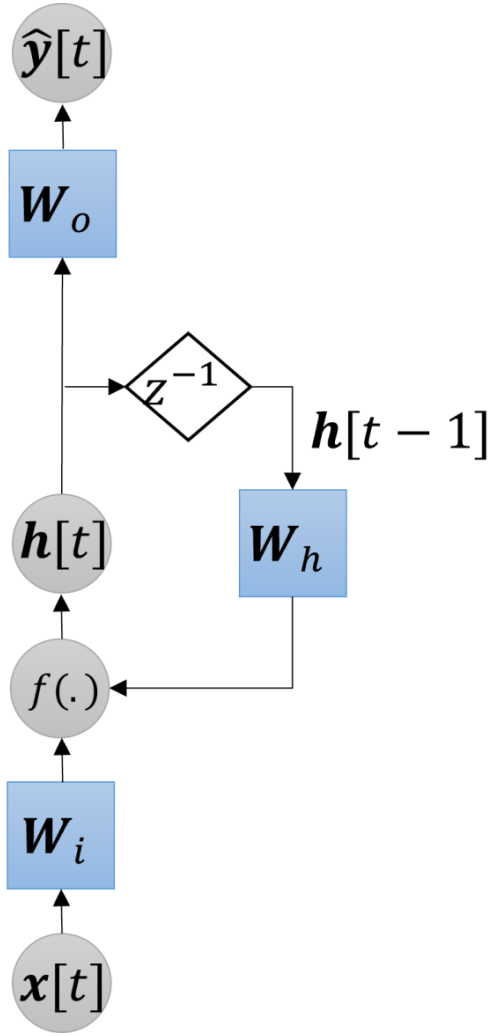
OBS.: Para facilitar a notação, usamos x_t ao invés de $x[t]$.

Neurônio recorrente



- A célula processa uma sequência de entradas um elemento (e.g., uma palavra ou um caractere) de cada vez, executando os seguintes passos em um laço:
  - Recebe a entrada atual da sequência, $\boldsymbol{x}_t$.
  - Recebe o estado oculto do passo de tempo anterior, $\boldsymbol{h}_{t-1}$ (sua "memória").
  - Combina essas duas entradas para calcular o novo estado oculto, $\boldsymbol{h}_t$.
  - Usa o novo estado oculto $\boldsymbol{h}_t$ para calcular a saída, $\boldsymbol{y}_t$ para o passo de tempo atual.
  - O novo estado oculto, $\boldsymbol{h}_t$, é então passado como memória para o próximo passo de tempo, $t + 1$.

Atualização do estado e geração da saída



- Equações de **atualização do estado oculto** e de **saída do neurônio** recorrente:

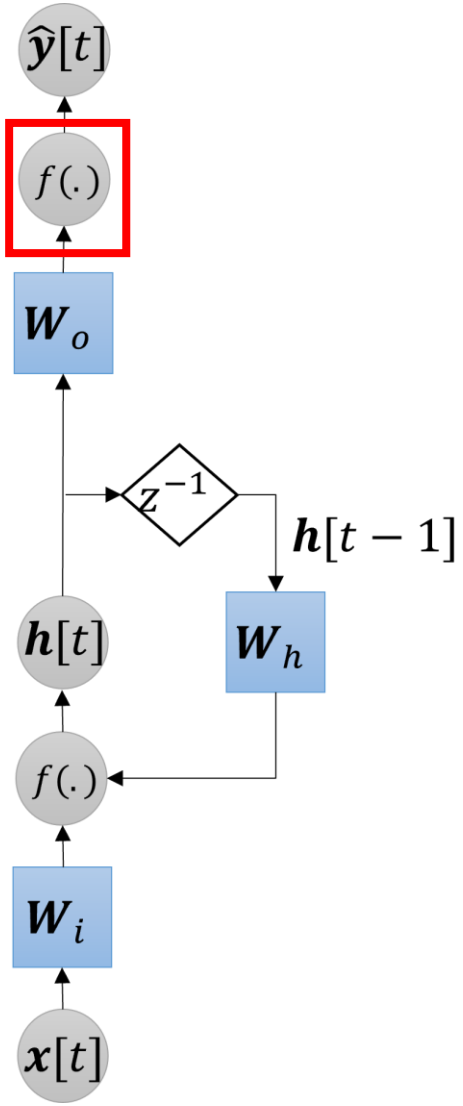
$$\boldsymbol{h}[t] = f(\boldsymbol{W}_h \boldsymbol{h}[t-1] + \boldsymbol{W}_i \boldsymbol{x}[t] + \boldsymbol{b}_h),$$

$$\hat{\boldsymbol{y}}[t] = \boldsymbol{W}_o \boldsymbol{h}[t],$$

onde $\boldsymbol{b}_h$ é o vetor de *bias*.

- Os **pesos são os mesmos** em **todos os intervalos de tempo**.
- Em geral, usa-se funções de ativação, $f(.)$, do tipo **tangente hiperbólica** (tanh) ou **sigmóide**.
  - Não se usa ReLU, pois **por não saturarem**, a **recorrência** pode **causar explosão do gradiente**.

Atualização do estado e geração da saída



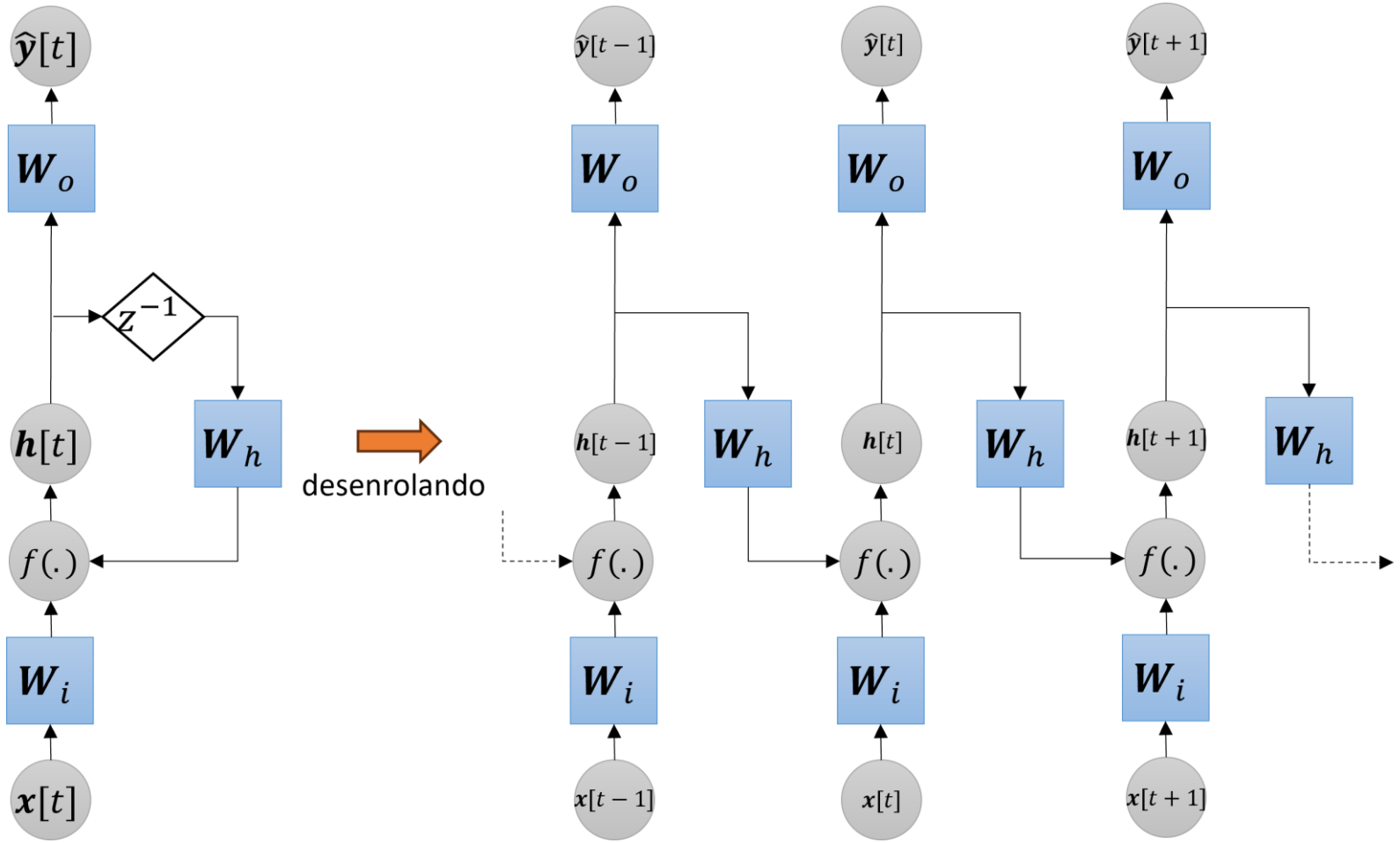
- Algumas implementações consideram uma **função de ativação na saída** (e.g., logística ou tanh).
- Desta forma, a equação de **saída do neurônio** recorrente fica assim:

$$\hat{\boldsymbol{y}}[t] = f(\boldsymbol{W}_o \boldsymbol{h}[t] + \boldsymbol{b}_o),$$

onde $\boldsymbol{b}_o$ é o vetor de *bias.*

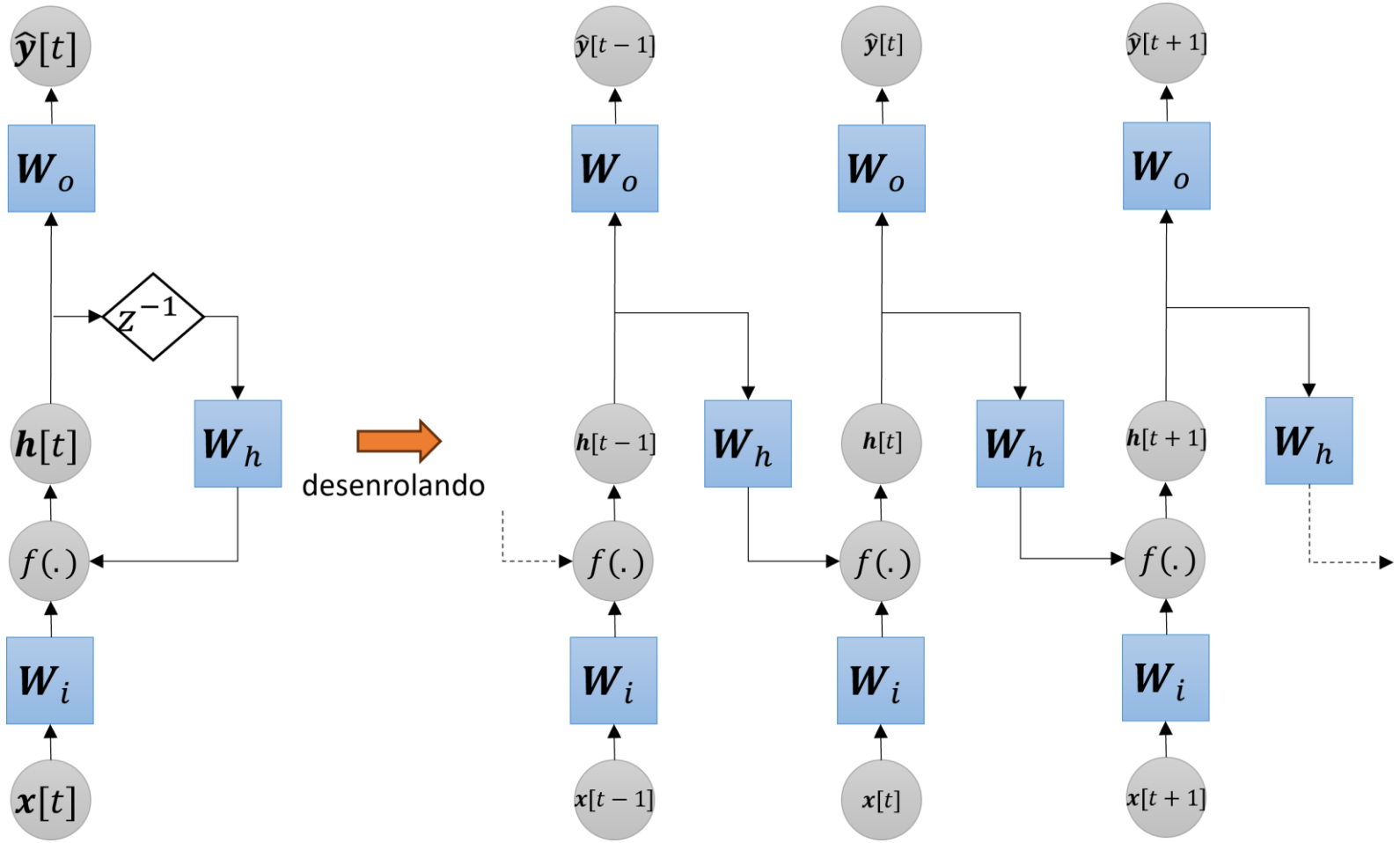
- A equação de **atualização do estado oculto** permanece a mesma.

Desenrolando uma RNN



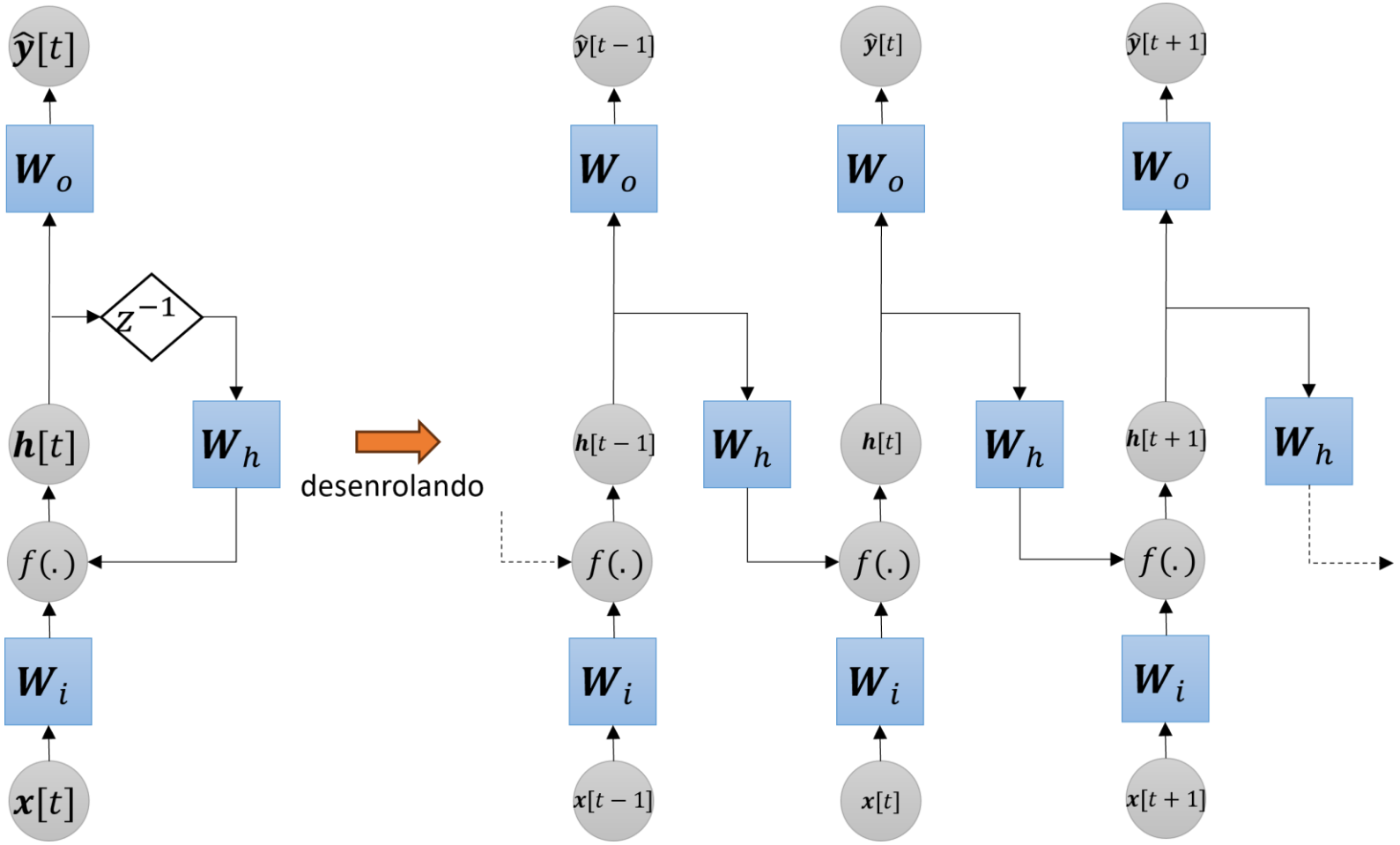
- Embora visualizemos a RNN com um **laço**, é mais fácil entender seu funcionamento **desenrolando-a** ao longo dos passos de tempo.
- Uma RNN desenrolada pode ser vista como **múltiplas cópias da mesma camada**, cada uma passando **uma mensagem para a sua sucessora**.

Desenrolando uma RNN



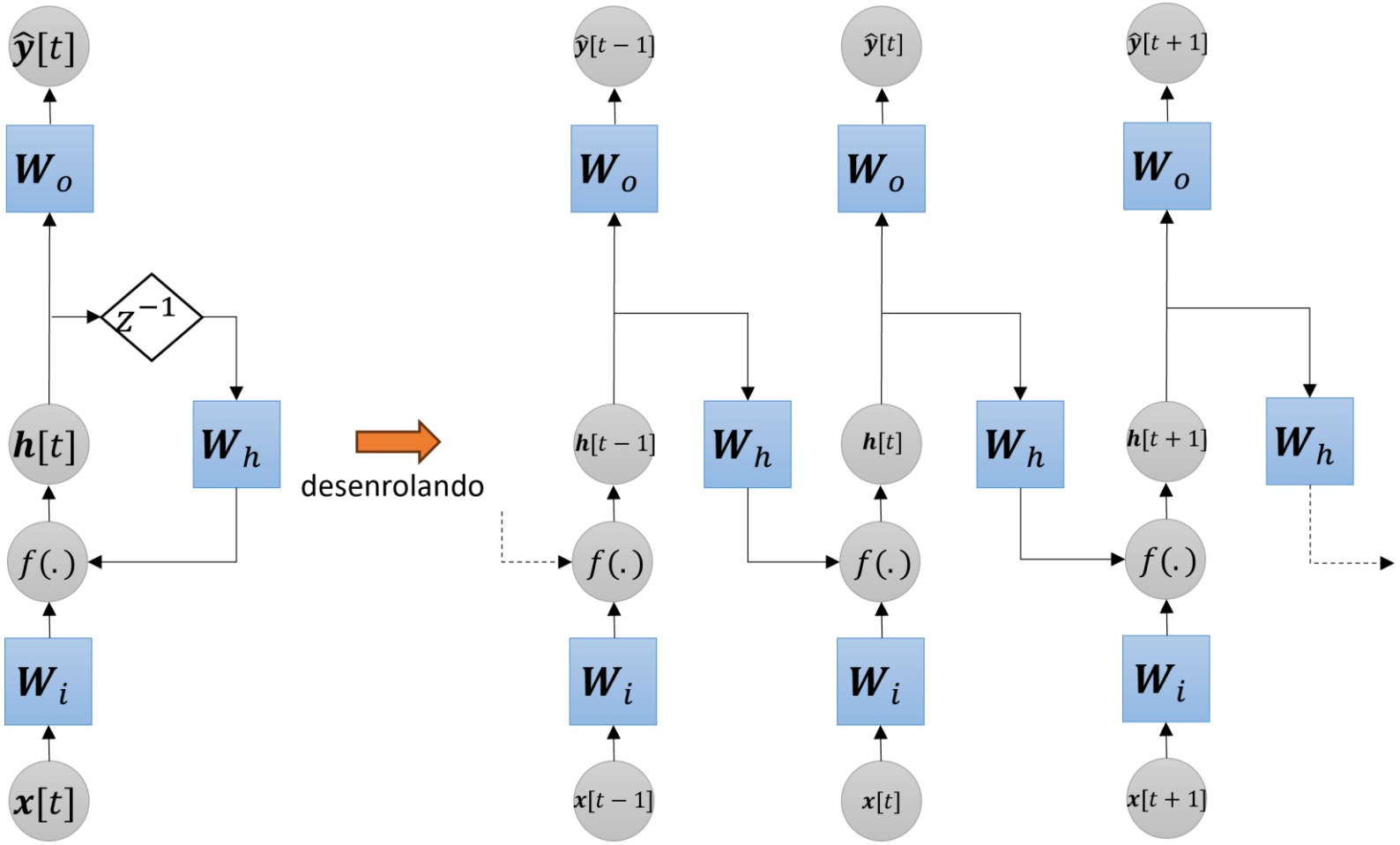
- Cada **réplica** da rede é **referente** a um **intervalo de tempo diferente** e pode ser pensada como **uma camada em uma rede MLP**.
- Percebam que a **arquitetura da rede** pode se tornar **muito profunda** para **sequências muito longas**.

Desenrolando uma RNN



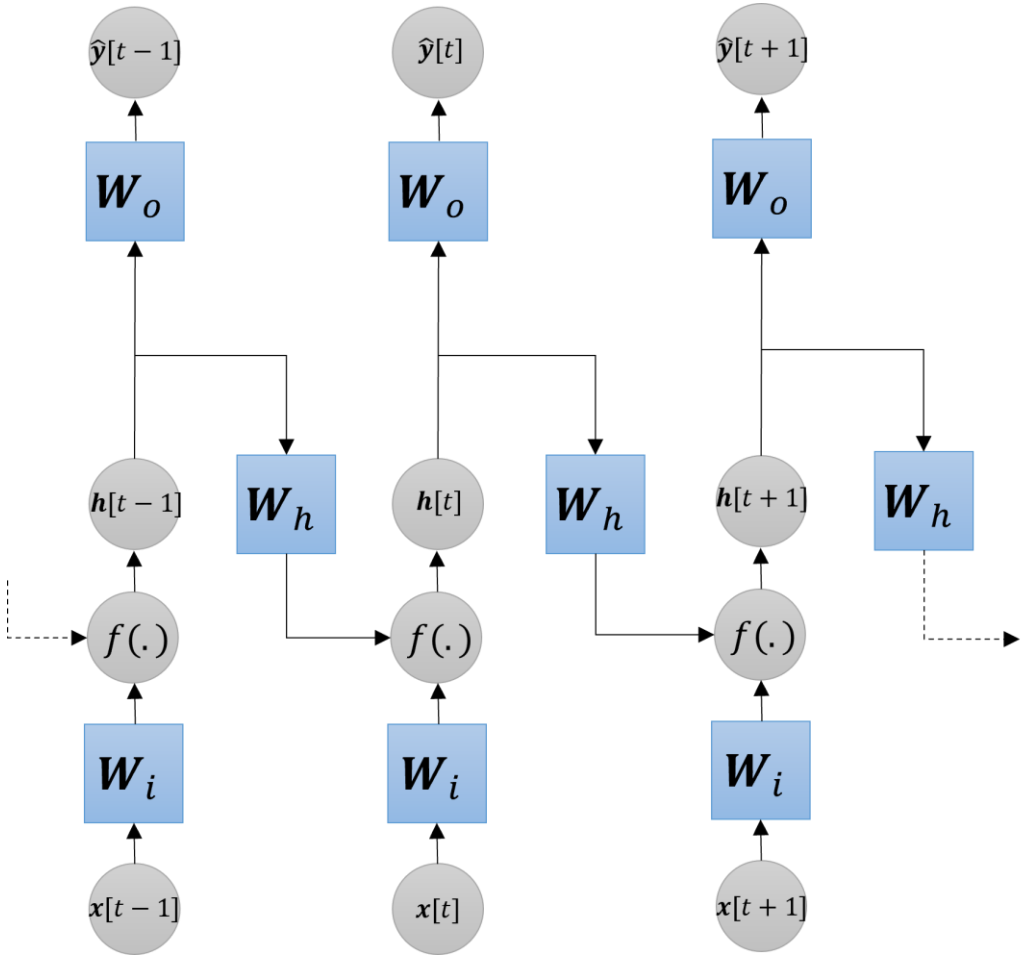
- Essa visão **desenrolada** mostra claramente o fluxo de informações e a **reutilização dos pesos**.
- Os **pesos** e as **funções de ativação** são **mantidos constantes** em cada **intervalo de tempo.**

Desenrolando uma RNN



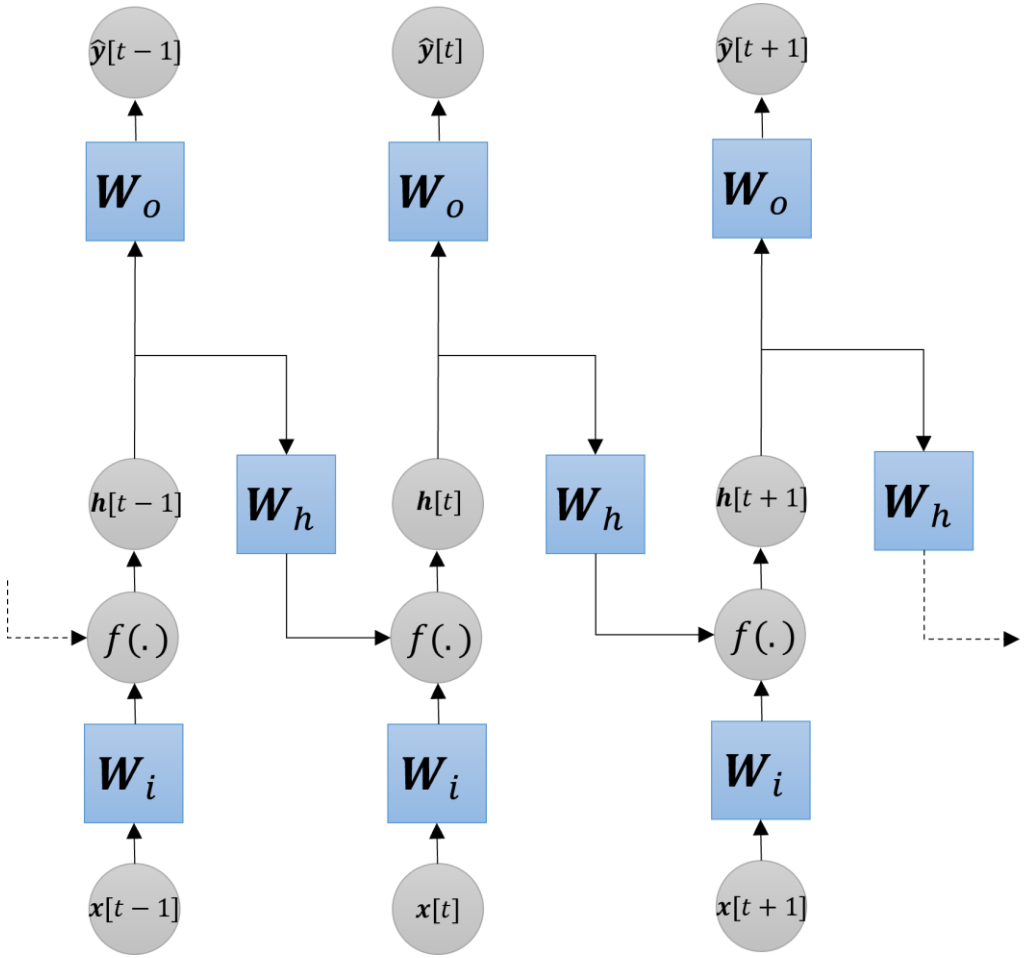
- Esse **conjunto compartilhado de pesos** é o que torna o modelo "**recorrente**" e permite que ele **aprenda (detecte) padrões em sequências**.
- Notem que a saída $\hat{y}[t]$ **depende de $h[t]$ e de todos os seus valores anteriores** e não apenas de $x[t]$ como em uma MLP.

Desenrolando uma RNN



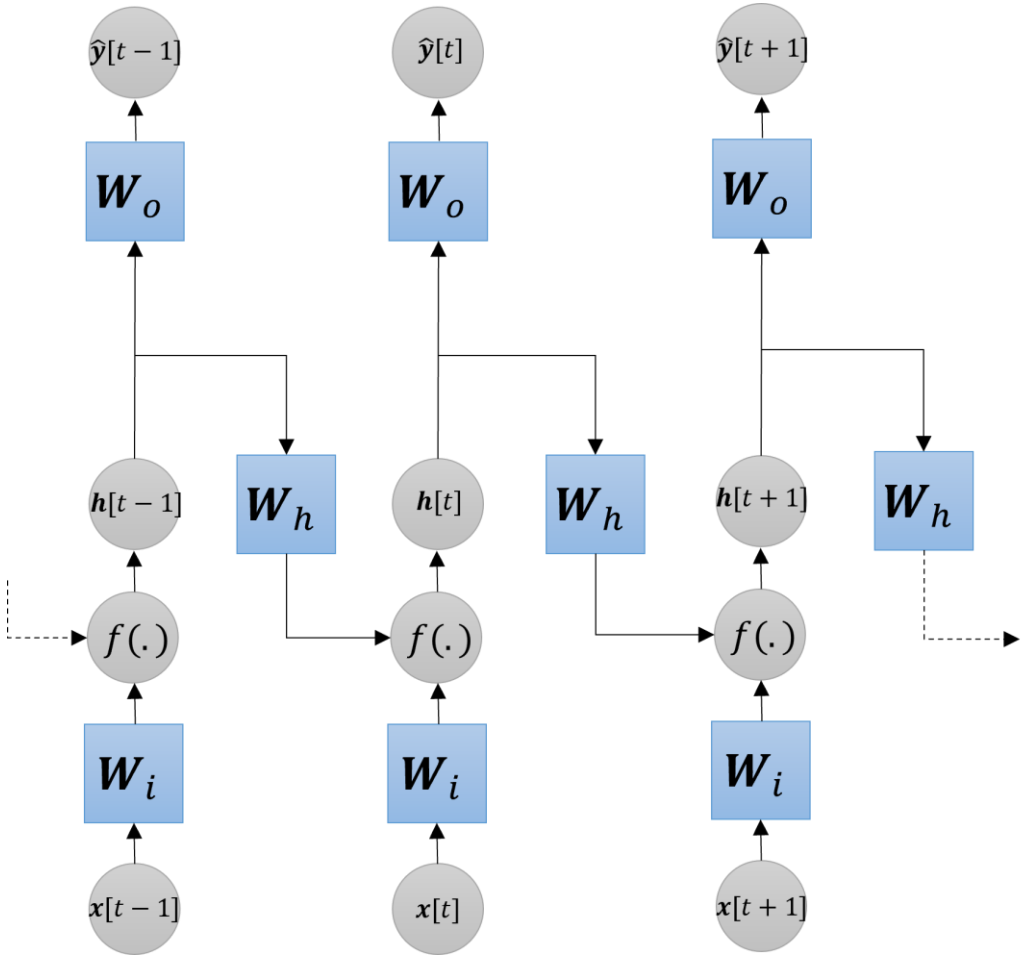
- O processo de desenrolar uma RNN nos ajuda a treinar a rede usando a **retropropagação do erro**.
- O objetivo é:
  - descobrir como cada peso influenciou o erro,
  - ajustar os pesos para minimizar o erro.
- Nas RNNs, isso é mais complexo pois:
  - o estado atual depende dos estados anteriores,
  - então o erro precisa voltar no tempo.

Treinando uma RNN



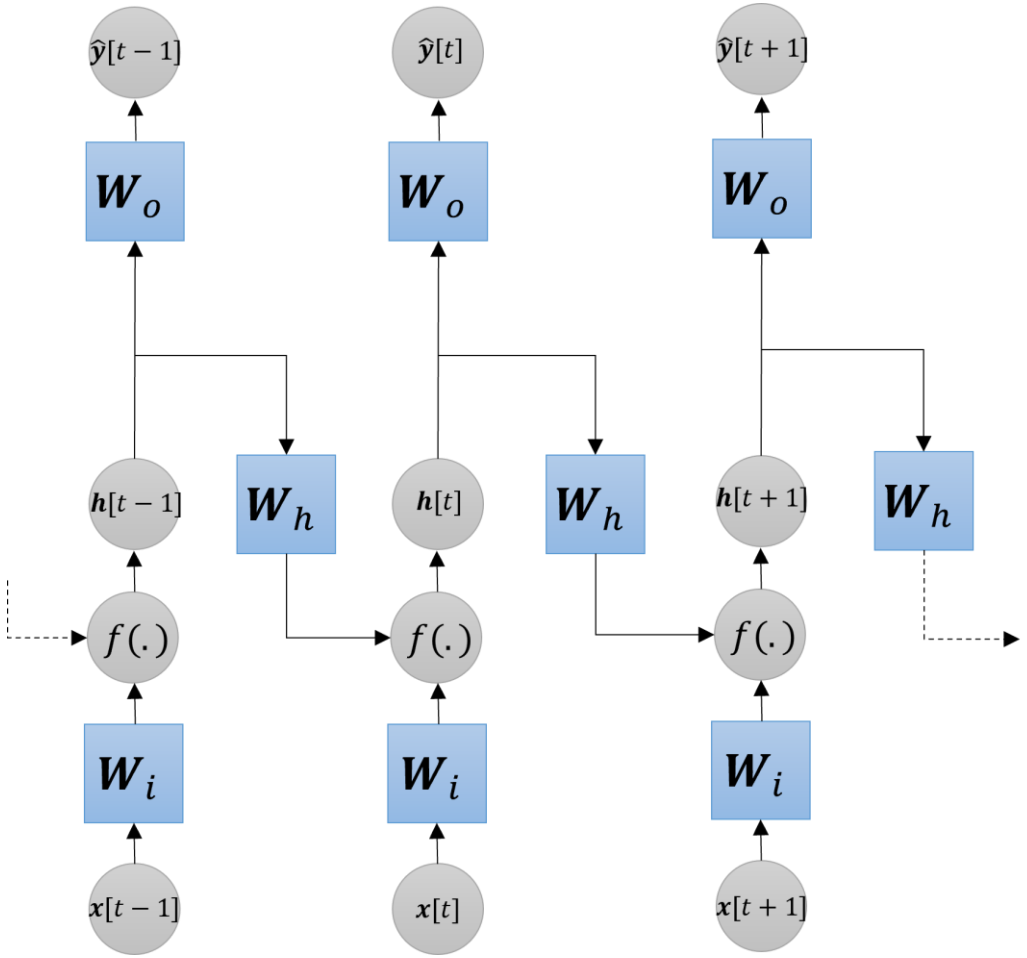
- Para treinar a rede, ela é primeiro **desenrolada** e **depois aplica-se a retropropagação** do erro **tradicional**.
- Inicialmente, a sequência, $(x_1, x_2, \ldots, x_T)$, é processada em uma **etapa direta**, gerando saídas ao longo dos vários instantes de tempo, $(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T)$.
- Em seguida, uma **função de erro**, E,(e.g., EQM, EC) é **calculada** utilizando as saídas e os rótulos, $(y_1, y_2, \ldots, y_T)$.

Treinando uma RNN



- Após o cálculo do erro, os **gradientes são propagados retroativamente** ao longo da rede desenrolada no tempo.
- Como os **mesmos pesos são reutilizados** em todos os instantes da sequência, **cada instante** temporal **produz uma contribuição para o gradiente final** desses pesos.
- Durante o treinamento, **essas contribuições são somadas** para gerar o **gradiente final** e atualizar os pesos da rede.

Treinando uma RNN

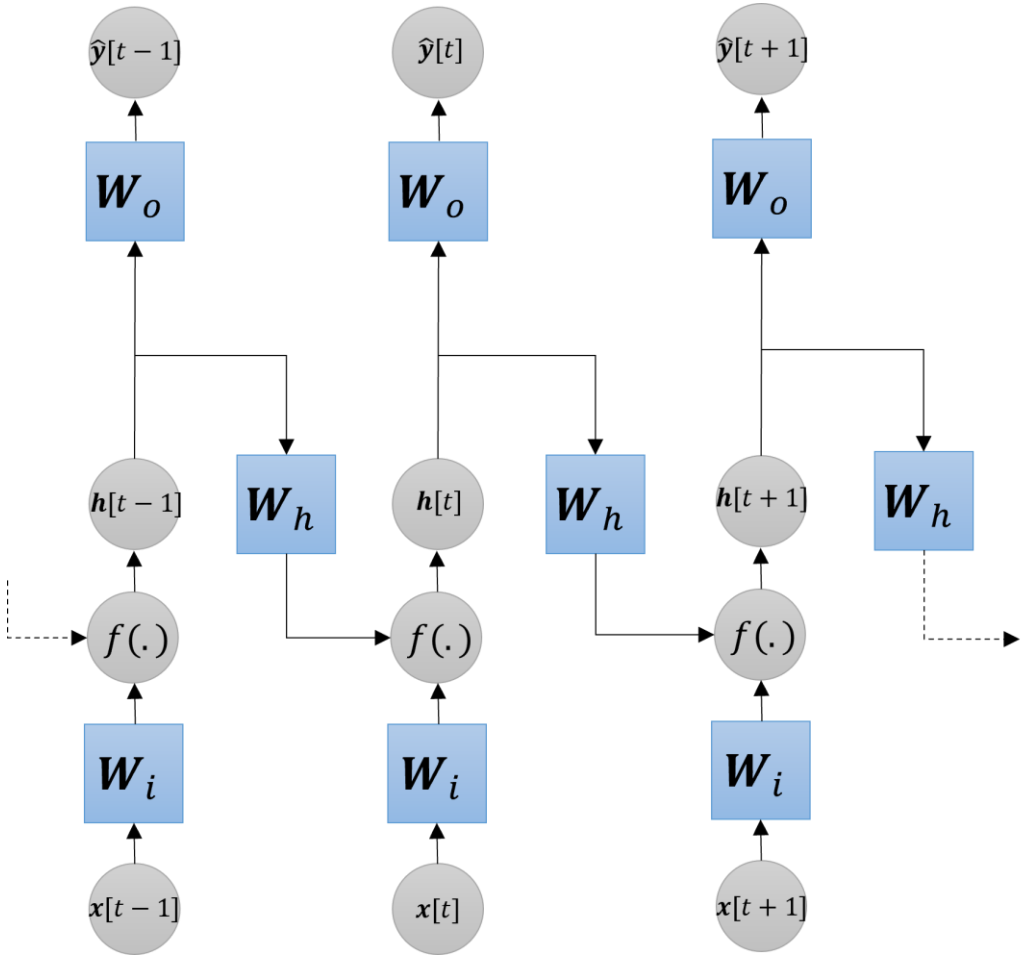


- Por fim, os pesos da rede são atualizados utilizando algum método de otimização baseados no vetor gradiente.
- Por exemplo, usando o gradiente descendente em batelada:

$$\boldsymbol{W} = \boldsymbol{W} - \alpha \frac{\partial E}{\partial \boldsymbol{W}},$$

onde $\boldsymbol{W}$ é uma das matrizes de peso da célula, α é o passo de aprendizagem e E é a função de erro.

Treinando uma RNN



- O **desenrolar da rede** para aplicação da retropropagação do erro é chamado de *Backpropagation Through Time* (BPTT).
- Ele é uma **extensão** do *backpropagation* tradicional, mas **aplicado ao longo do tempo**.
- A ideia central é:

  o **erro atual** pode ter **sido causado por estados passados** da rede.
- Então o gradiente precisa "**voltar no tempo**".

Backpropagation Through Time (BPTT)

- O objetivo do treinamento é minimizar a **erro total** gerado pela rede **ao longo de todos os passos de tempo**.
- Inicialmente, o **erro instantâneo** é calculado para cada passo de tempo, t, da sequência temporal

$$E_t = e(y[t], \hat{y}[t]).$$

- O **erro total** é a média do **erro instantâneo** para todos os instantes de tempo considerados, T

$$E = \frac{1}{T}\sum_{t=0}^{T-1} E_t = \frac{1}{T}\sum_{t=0}^{T-1} e(y[t], \hat{y}[t])\,.$$

- Em outras palavras, o **erro total** é uma **média temporal dos erros** cometidos pela rede **ao longo de todos os T instantes de tempo**.

Backpropagation Through Time (BPTT)

- De forma genérica, a derivada do erro em relação à matriz de pesos, $\boldsymbol{W}$ é obtida usando a **regra da cadeia**:

$$\frac{\partial E}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial E_t}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial e(y[t],\hat{y}[t])}{\partial \hat{y}[t]}\frac{\partial \hat{y}[t]}{\partial h[t]}\frac{\partial h[t]}{\partial \boldsymbol{W}}.$$

onde $h[t] = f(x[t], h[t-1], \boldsymbol{W})$ é o estado oculto.

- O gradiente é obtido somando-se as **contribuições de cada passo de tempo.**
- Como os pesos são compartilhados ao longo do tempo, uma única atualização de peso deve levar em conta sua influência na predição (e, consequentemente, no erro) feita em $t = 0$, $t = 1$ até $t = T - 1$.
- Em outras palavras, o BPTT propaga os erros do futuro para o passado.

Backpropagation Through Time (BPTT)

$$\frac{\partial E}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial E_t}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial e(y[t],\hat{y}[t])}{\partial \hat{y}[t]}\frac{\partial \hat{y}[t]}{\partial h[t]}\boxed{\frac{\partial h[t]}{\partial \boldsymbol{W}}}.$$

- Os dois primeiros termos são fáceis de se calcular, mas o terceiro é onde as coisas se complicam, já que precisamos calcular **recursivamente** o efeito da matriz de pesos $\boldsymbol{W}$ sobre $h[t]$.
- $\boldsymbol{W}$ afeta o erro E_t de duas maneiras:
  - **Diretamente:** No cálculo do estado oculto atual $h[t]$.
  - **Indiretamente:** Porque $h[t]$ depende de $h[t-1]$, que por sua vez dependeu de $\boldsymbol{W}$ no passado, que dependeu de $h[t-2]$, e assim por diante, voltando até o estado inicial $h[0]$.

Backpropagation Through Time (BPTT)

- Aplicando a regra da cadeia ao terceiro termo, obtemos a formulação analítica completa do BPTT para o passo de tempo t:

$$\frac{\partial E}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial E_t}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{t}\frac{\partial e(y[t],\hat{y}[t])}{\partial \hat{y}[t]}\frac{\partial \hat{y}[t]}{\partial h[t]}\left(\prod_{j=k+1}^{t}\frac{\partial h[j]}{\partial h[j-1]}\right)\frac{\partial h[k]}{\partial \boldsymbol{W}}.$$

- Notem que o gradiente envolve um **produto de muitas derivadas sucessivas**.
- O **produtório** na equação acima **pode causar o desaparecimento ou a explosão do gradiente** dependendo dos valores das derivadas parciais, $\frac{\partial h[j]}{\partial h[j-1]}$.

Desaparecimento e explosão do gradiente

- Se $\frac{\partial h[j]}{\partial h[j-1]} \forall j < 1$, então o produto tende a **zero**
  - A rede **esquece o passado mais distante** (informação antiga se perde).
  - Não aprende **dependências de longo prazo, apenas curtas.**
    - Estados ocultos anteriores não têm efeito sobre os estados posteriores.
  - Treinamento se torna **muito lento** ou fica **estagnado**.
- Se $\frac{\partial h[j]}{\partial h[j-1]} \forall j > 1$, então o produto **explode**
  - O modelo **diverge**.
  - Atualizações de pesos se tornam enormes.
  - **Treinamento instável**: erro vira NaN.
  - Sintoma típico: **oscilação do erro**.
- Na prática, o desaparecimento do gradiente é mais comum.

Como mitigar esses problemas?

- **Para a explosão**:
  - Limitar a magnitude do gradiente (*gradient clipping*)
  - Inicializar os pesos de forma apropriada (inicializações de He e Xavier)
- **Para o desaparecimento**:
  - Usar arquiteturas de RNNs mais avançadas: LSTM e GRU
    - Introduzem o **conceito de *gates***, funcionando como **válvulas de informação e gradiente**
      - ✓ Eles são funções (em geral, sigmoide) que produzem valores entre 0 e 1:
        - ➢ 0 → bloqueia informação
        - ➢ 1 → deixa passar totalmente
    - Criam caminhos com derivada ≈ 1
      - ✓ **Mantendo o gradiente estável**
  - **Conexões residuais**: criam **caminhos diretos** para o gradiente
  - Inicializar os pesos de forma apropriada (inicializações de He e Xavier)

Backpropagation Through Time (BPTT)

$$\frac{\partial E}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\frac{\partial E_t}{\partial \boldsymbol{W}} = \frac{1}{T}\sum_{t=0}^{T-1}\sum_{k=1}^{t}\frac{\partial e(y[t],\hat{y}[t])}{\partial \hat{y}[t]}\frac{\partial \hat{y}[t]}{\partial h[t]}\left(\prod_{j=k+1}^{t}\frac{\partial h[j]}{\partial h[j-1]}\right)\frac{\partial h[k]}{\partial \boldsymbol{W}}.$$

- Na prática, essa equação diz que para atualizar a matriz de pesos $\boldsymbol{W}$, a rede
  - calcula o erro em cada saída,
  - propaga esse erro de volta através das conexões recorrentes até o início da sequência
  - e acumula os ajustes baseados nos estados passados.
- Se o valor de T for grande (i.e., uma **sequência muito longa**), temos uma **profundidade intratável**, o que torna o **treinamento muito lento**, **consome muita memória** e pode deixar o **treinamento instável devido à recursão**.

Backpropagation Through Time (BPTT)

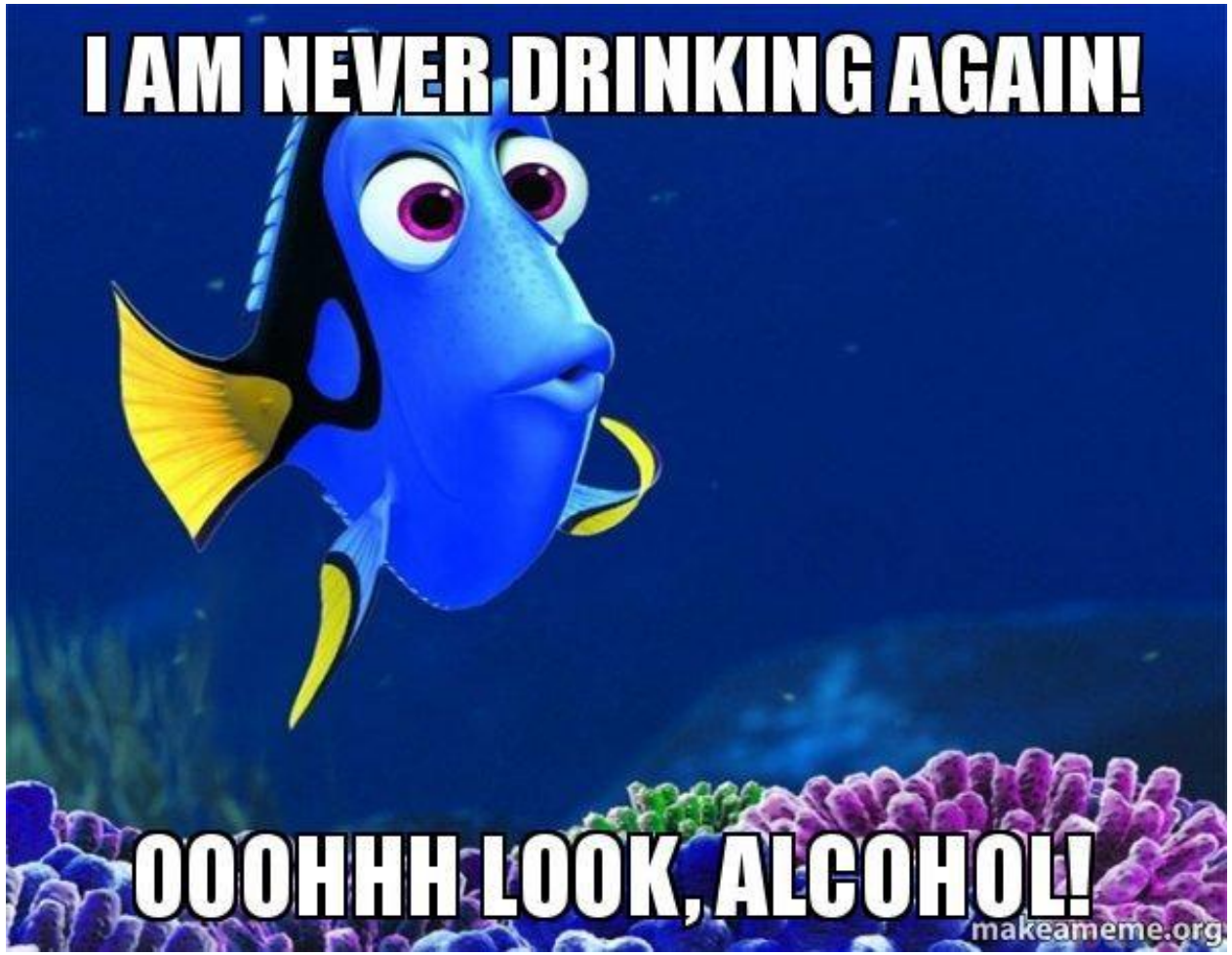
- Na prática, muitas vezes **não propagamos até o início da sequência**, mas até um determinado instante de tempo, τ.
  - Esse processo é chamado de **BPPT truncado (TBPPT)**.
  - τ é um hiperparâmetro crucial que deve ser otimizado: pequeno demais prejudica a memória de longo prazo e grande demais perde a eficiência que motivou a técnica.
- Ele reduz:
  - custo computacional,
  - uso de memória.
  - e ajuda a combater os problemas de explosão e desaparecimento do gradiente.
- Porém, o TBPPT força a rede a se concentrar em padrões de curto e médio alcance.
- Dependências que excedem o tamanho do truncamento **não são aprendidas.**

Por que o BPTT truncado funciona?

- Funciona porque, na prática, **muitas dependências temporais relevantes são locais**.
- Além disso, muitas **dependências temporais relevantes decaem com o tempo**.
- Isso **evita** a necessidade de **voltar até o primeiro passo** da sequência **para calcular o gradiente**.
- Ou seja:
  - o **estado atual** geralmente depende mais do **passado recente**,
  - **contribuições** muito **antigas** tendem a **ficar pequenas**.

Por que não usar funções de ativação ReLU?

- RNNs **reusam** a **mesma matriz de pesos e a mesma função de ativação repetidamente** em cada passo.
- **Valores maiores que 1** na entrada de uma **função que não satura** como a ReLU ($\boldsymbol{y} = \max(\boldsymbol{x}, 0)$ podem causar um crescimento exponencial (explosão) dos valores do estado oculto, $h[t]$.
- Isso torna o treinamento instável:
  - Os estados podem crescer sem limite, causando dinâmica instável no tempo.
  - Em **sistemas dinâmicos** (como em RNNs), isso é ruim, pois **causa oscilações ou divergência**.
- Além disso, **valores negativos** em sua entrada **geram saídas e derivadas iguais a 0**, fazendo com que o **fluxo de aprendizado morra**.



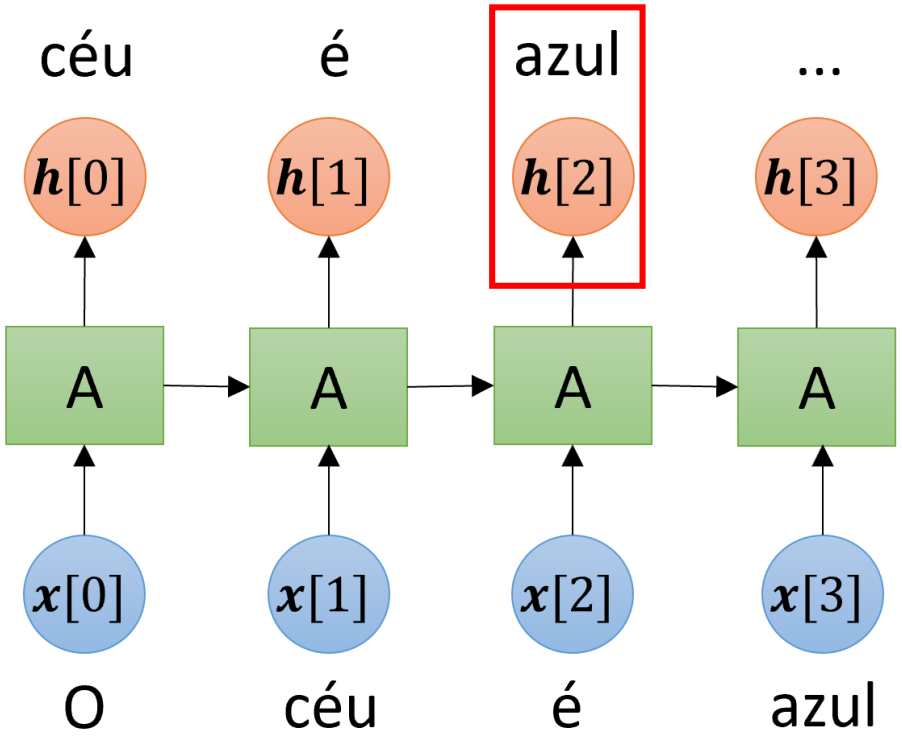
Quando uma **RNN tradicional** processa uma **sequência muito longa**, ela se **esquece gradualmente das primeiras amostras** da sequência.

Ou seja, elas **não aprendem dependências longas**

O problema das dependências de longo prazo

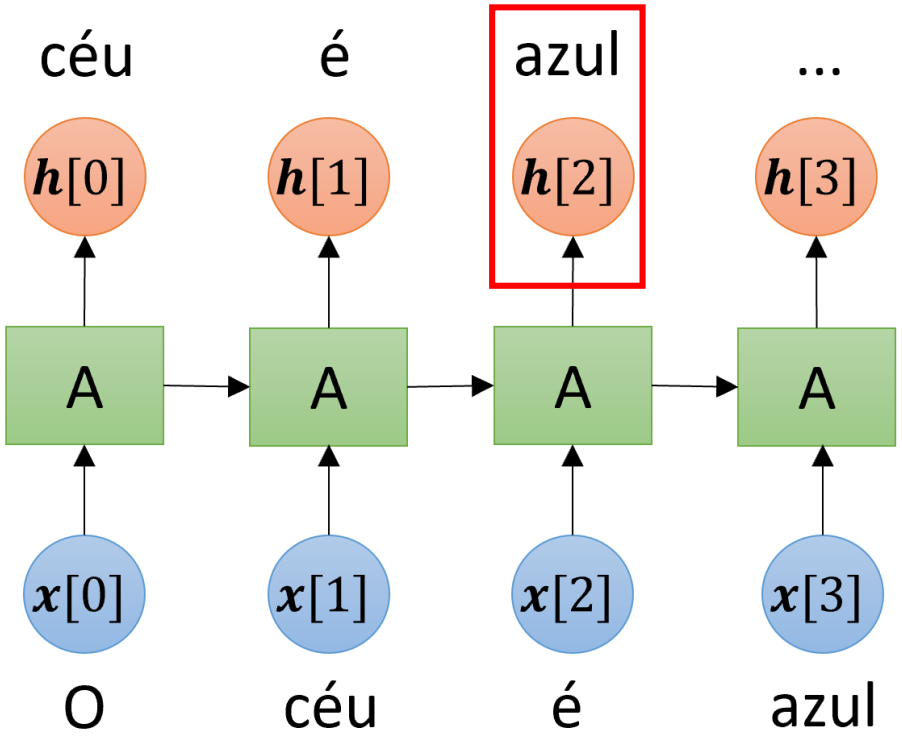
- Devido ao **desaparecimento do gradiente**, as RNNs tradicionais (baunilha) treinadas usando o BPTT **são incapazes de aprender dependências de longo prazo**.
- O **BPTT truncado** **também limita o aprendizado de longo prazo**, pois **impõe um horizonte** máximo **para a retropropagação** dos gradientes.
- Entretanto, **algumas aplicações exigem dependências tanto de curto quanto de longo prazo**.
- Por exemplo, em tradução automática, o significado de uma palavra ou frase pode depender de palavras vizinhas ou de um contexto que apareceu muito antes no texto.

O problema das dependências de longo prazo



- Considere treinar a RNN ao lado para prever **a próxima palavra**.
- Para **prever** a palavra **"azul"**, a RNN precisa apenas de **informação de dois passos atrás**.
- Ou seja, a **informação relevante está a poucos passos** no tempo (**contexto recente**).

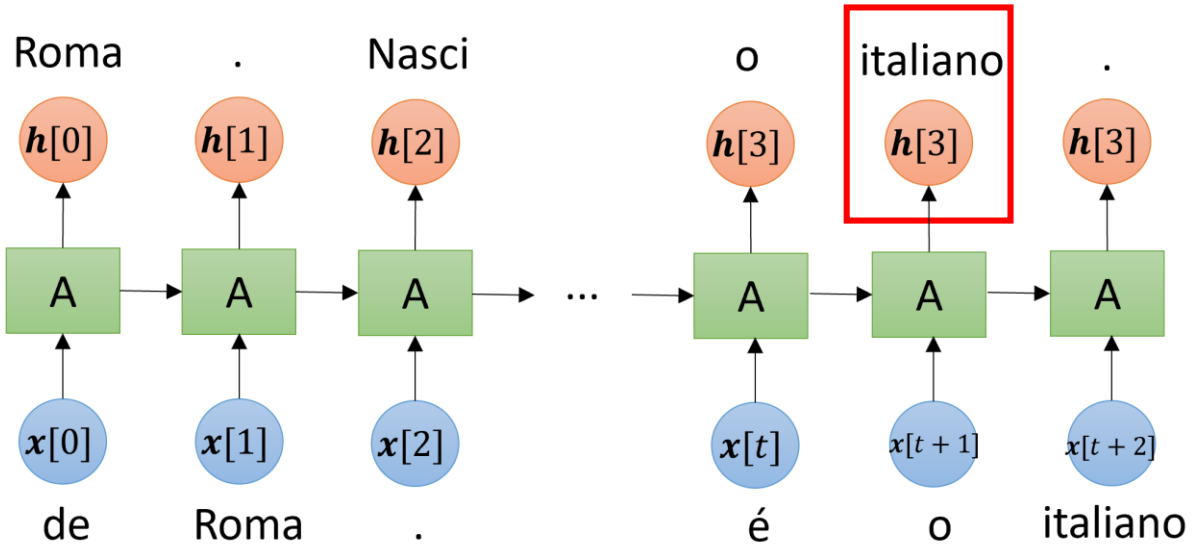
O problema das dependências de longo prazo



- Assim, durante o **treinamento**, o **erro precisa ser retropropagado por apenas alguns instantes** temporais.
- Porém, em muitas aplicações, a **informação importante pode estar muito distante na sequência**.
- Agora, vamos considerar a frase:

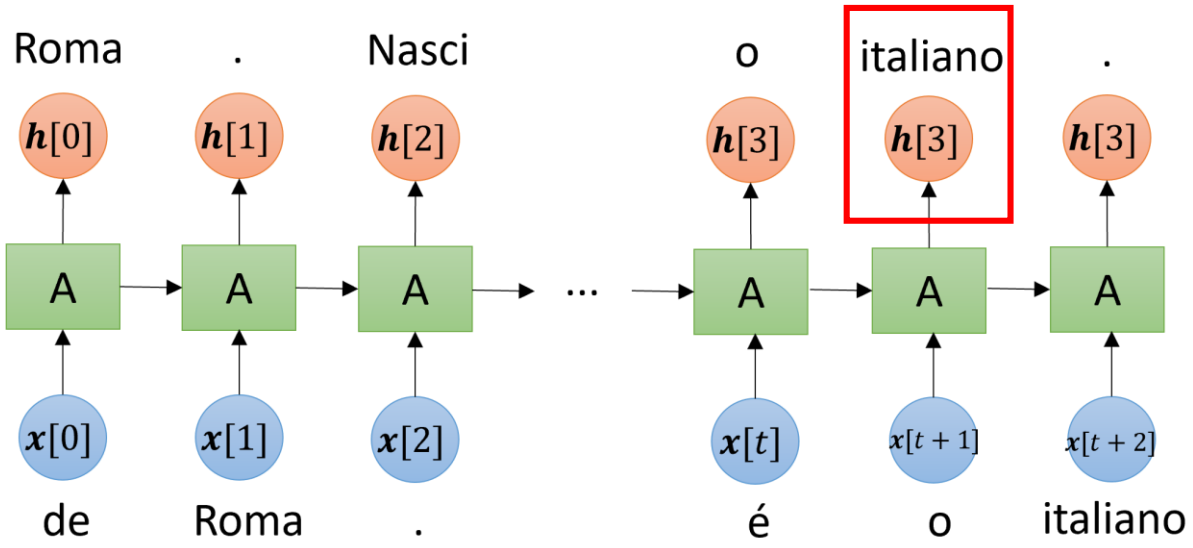
  "Sou de **Roma**. Nasci há 30 anos. Gosto de comer bem e andar de bicicleta. Minha língua materna é o **italiano**."

O problema das dependências de longo prazo



- Para prever **"italiano"**, a rede precisa lembrar de uma **informação muito antiga** da sequência, a palavra **"Roma"**.
- Durante o treinamento, o **erro** precisa ser **retropropagado** por **vários instantes de tempo**, o que pode fazer com que o **gradiente desapareça**.

O problema das dependências de longo prazo



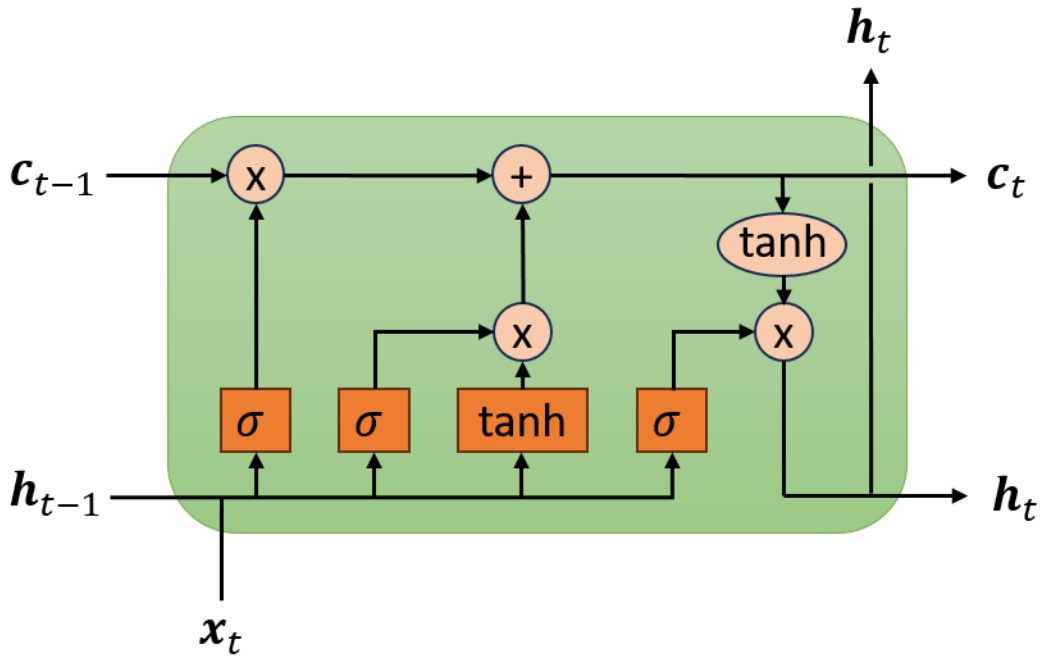
- **Quanto maior a distância temporal**, **mais difícil é para uma RNN tradicional** manter a informação relevante, ou seja, **se lembrar do passado distante**.

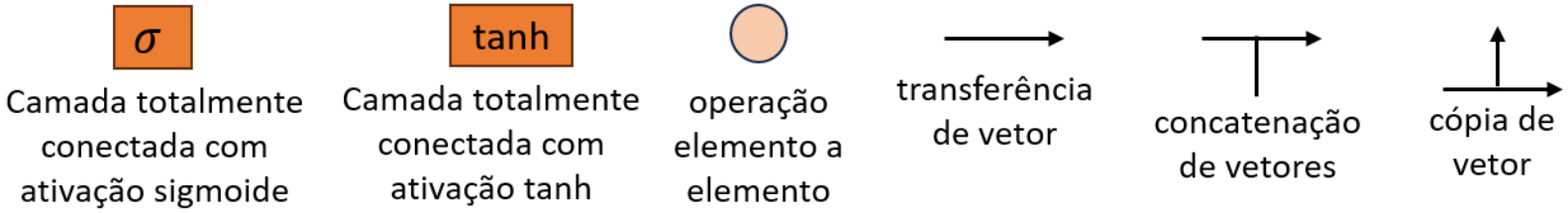
Como permitir que a rede preserve **informações importantes** **por muito mais tempo**?

O problema das dependências de longo prazo

- Para **diminuir o problema das dependências de longo prazo**, causado pelo desaparecimento ou explosão do gradiente, foram propostas **arquiteturas com mecanismos de memória**:
  - Long Short-Term Memory (LSTM)
  - Gated Recurrent Unit (GRU)
- Essas redes **criam caminhos ao longo do tempo** cujas **derivadas podem ser controladas** para não desaparecer nem explodir.
- Elas utilizam **portas (*gates*)** para **controlar** quais **informações** devem ser:
  - armazenadas (o que lembrar),
  - esquecidas ou mantidas,
  - e expostas na saída (quando usar uma informação armazenada).

Long-short term memory (LSTM)

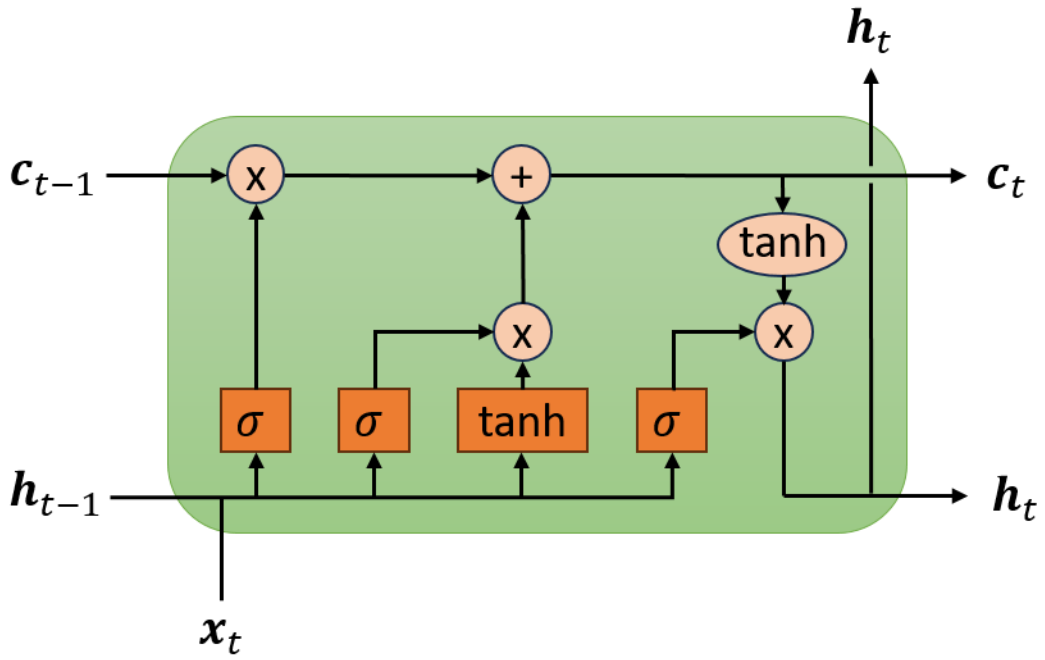


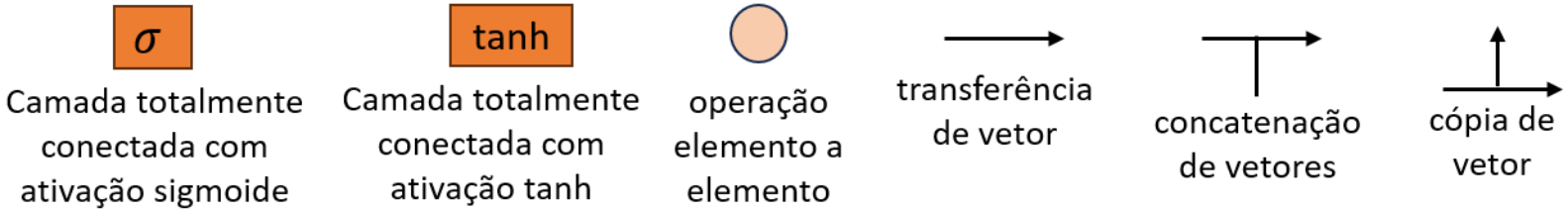


- Introduzida por Hochreiter e Schmidhuber em 1997.
- Funcionam muito bem em **diversos problemas que envolvem dados sequenciais**.
- São amplamente utilizadas atualmente.

OBS.: Para facilitar a notação, usamos x_t ao inves de $x[t]$.

Long-short term memory (LSTM)

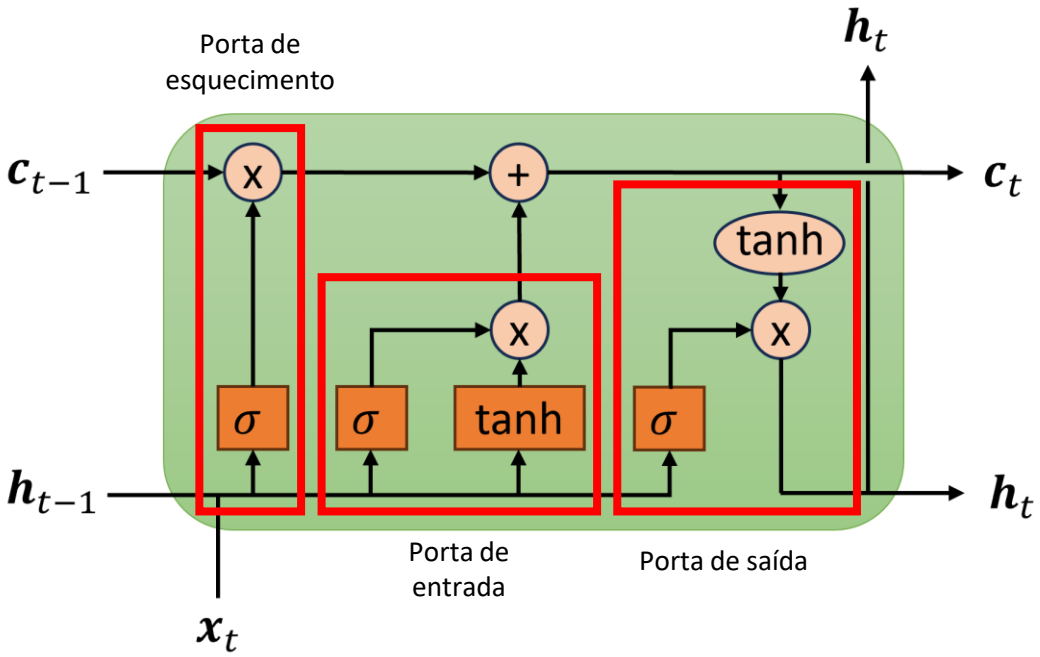




- Elas **capturam padrões de longo e curto prazo** de uma sequência de dados.
- Assim como as RNNs tradicionais, elas **precisam ser desdobradas** no tempo para serem **treinadas**.

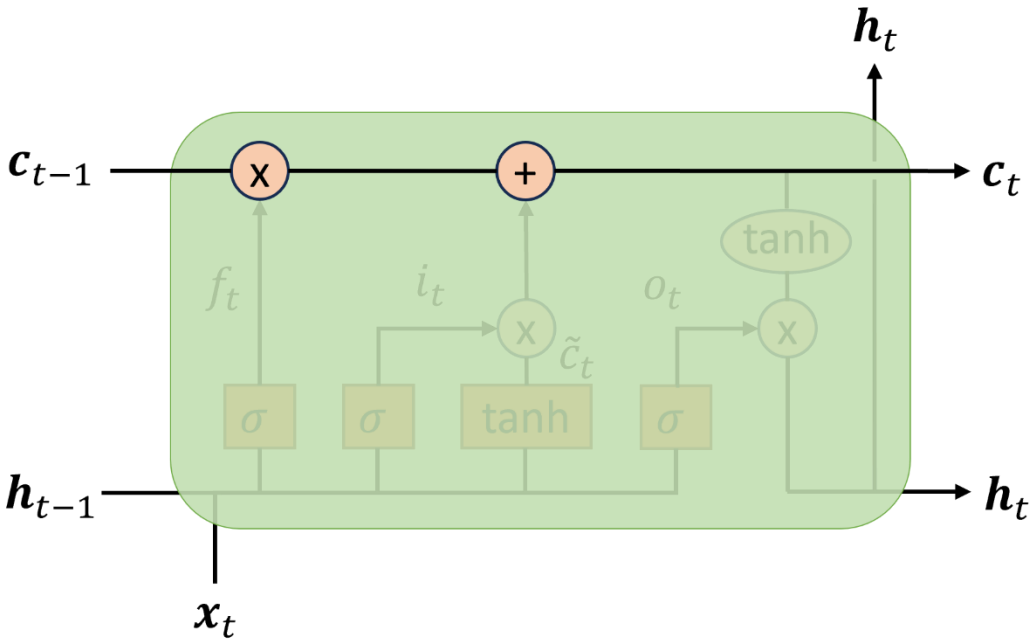
OBS.: Para facilitar a notação, usamos x_t ao inves de $x[t]$.

Long-short term memory (LSTM)



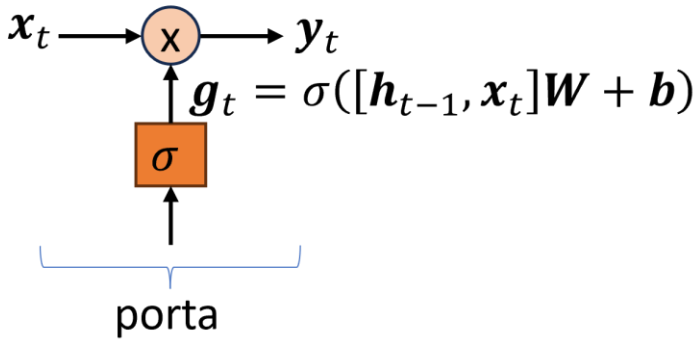
- A **unidade de processamento** (neurônio) da LSTM é mais **complexa** e é chamada de **célula (de memória)**.
- Uma célula LSTM é composta por **4 camadas totalmente conectadas**, que formam **3 portas**:
  - ***Forget gate*** (porta de esquecimento)
  - ***Input gate*** (porta de entrada)
  - ***Output gate*** (porta de saída)
- A informação na célula é **protegida e controlada** por meio destas 3 portas.

Long-short term memory (LSTM)



- Uma célula LSTM possui dois estados:
  - o estado interno da célula, $\boldsymbol{c_t}$
  - e o estado oculto da célula, $\boldsymbol{h_t}$.
- $\boldsymbol{c_t}$ é a memória de longo prazo.
- $\boldsymbol{h_t}$ é memória de curto prazo e a saída da célula.

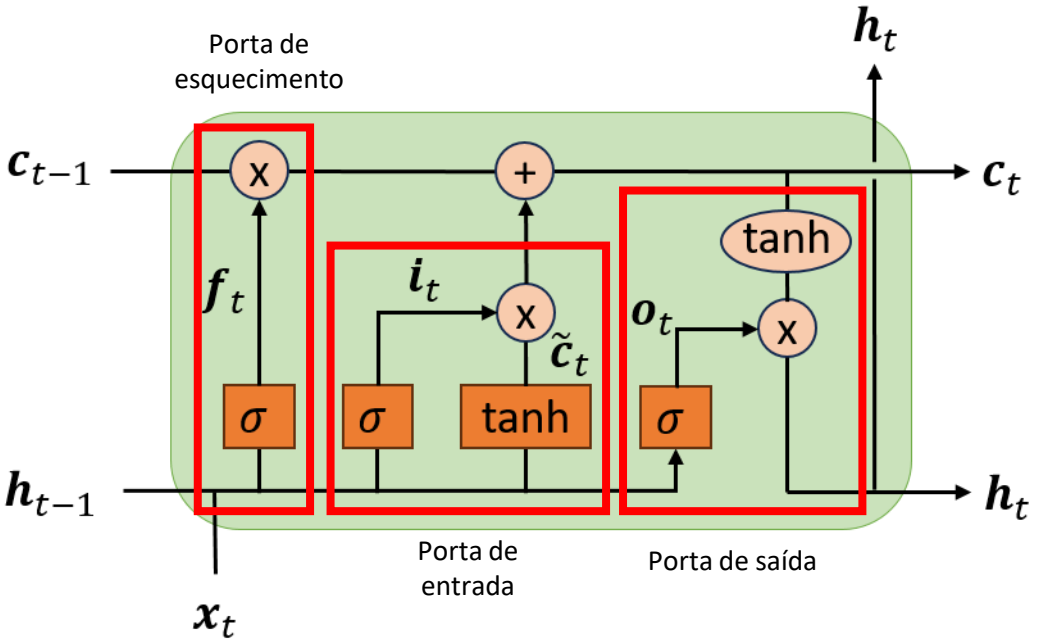
Como as portas funcionam?



x_t		g_t		y_t
0.58		0.2		0.116
-0.32	x	0.99	=	-0.3168
0.1		0.01		0.001
1.24		0.95		1.178

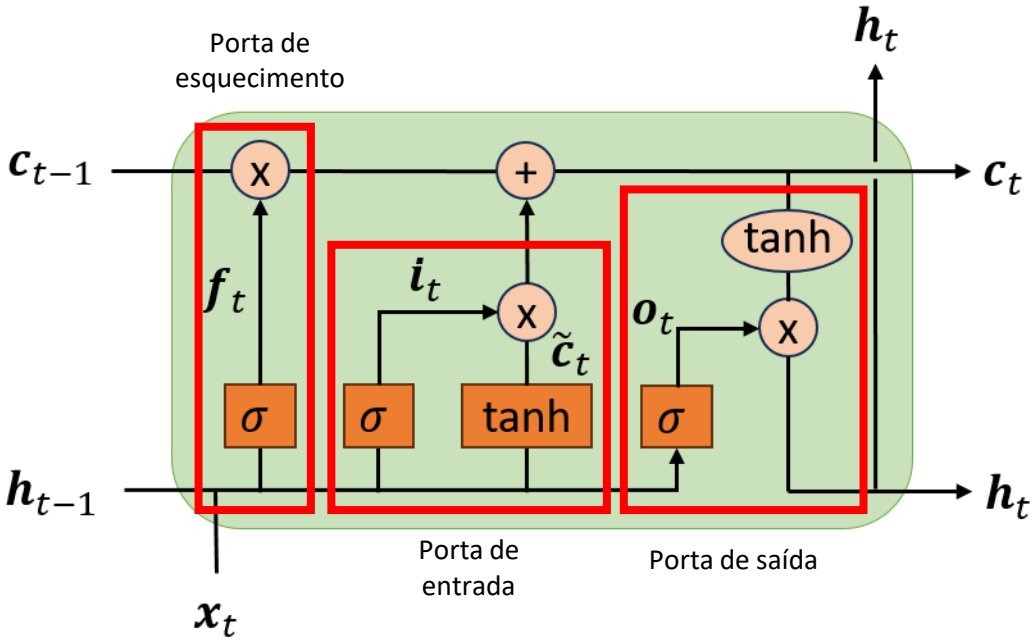
- As portas são uma forma de **permitir** a **passagem ou não de informações**.
- Imaginem uma das 3 camadas com função de ativação sigmoide da célula LSTM.
- A camada gera um **vetor**, g_t, com **elementos assumindo valores entre 0 e 1**.
- Uma **operação de multiplicação elemento a elemento** decide **o quanto da entrada**, x_t passa para a saída, y_t.
- A porta **aprende** (através dos **pesos**) o quanto de cada elemento deve ser permitido passar.

Long-short term memory (LSTM)



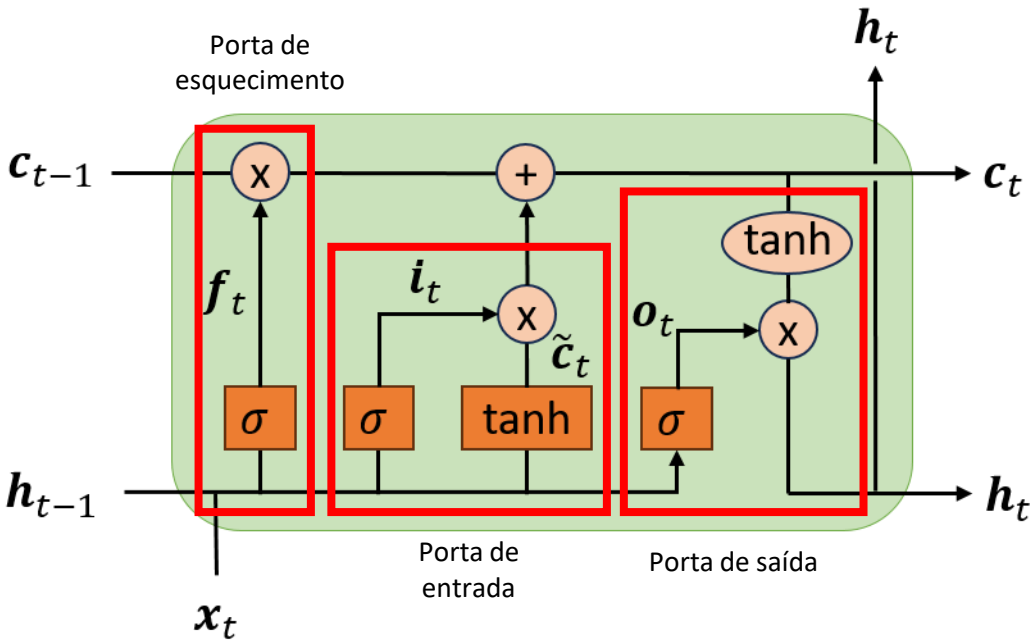
- A **porta de esquecimento** (controlada pelo vetor $\boldsymbol{f}_t$) controla **quais partes** da **informação passada**, $\boldsymbol{c}_{t-1}$, devem ser **apagadas ou mantidas**.
- A **porta de entrada** (controlada pelo vetor $\boldsymbol{i}_t$) controla **quais partes** de **novas informações (memórias)**, $\tilde{\boldsymbol{c}}_t$, devem ser **adicionadas** à $\boldsymbol{c}_{t-1}$.
- A **porta de saída** (controlada pelo vetor $\boldsymbol{o}_t$) controla **quais partes** de $\boldsymbol{c}_t$ devem ser **expostas na saída**, $\boldsymbol{h}_t$.

Como as portas funcionam?



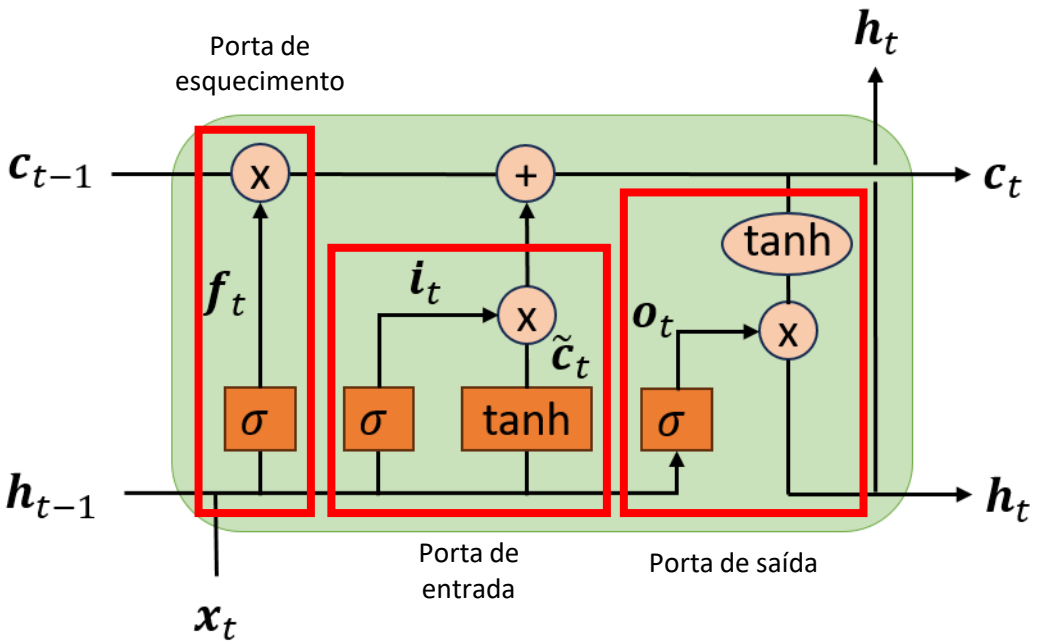
- **Porta de entrada**: uma **nova informação** é **adicionada** à **memória de longo prazo**, c_{t-1}, sempre que **um elemento de i_t tender a 1**.
- **Porta de esquecimento**: uma **informação de longo prazo** é **descartada** da **memória de longo prazo**, c_{t-1}, sempre que **um elemento de f_t tender a 0**.

Como as portas funcionam?



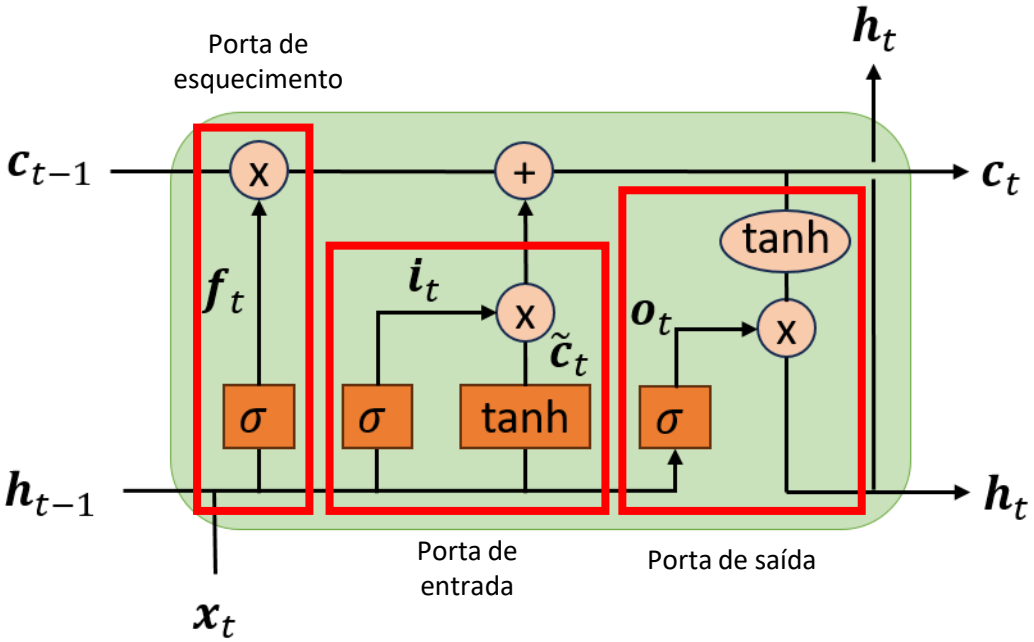
- **Porta de saída**: **uma informação de longo prazo** é **exposta** na **saída da célula** sempre que **um elemento de o_t tender a 1**.
- Em resumo, uma **célula** LSTM **aprende a reconhecer** uma **informação importante**, **armazená-la** no estado de longo prazo, **preservá-la** pelo tempo que for necessário e **expô-la** sempre que necessário.

Dinâmica das portas em uma célula LSTM



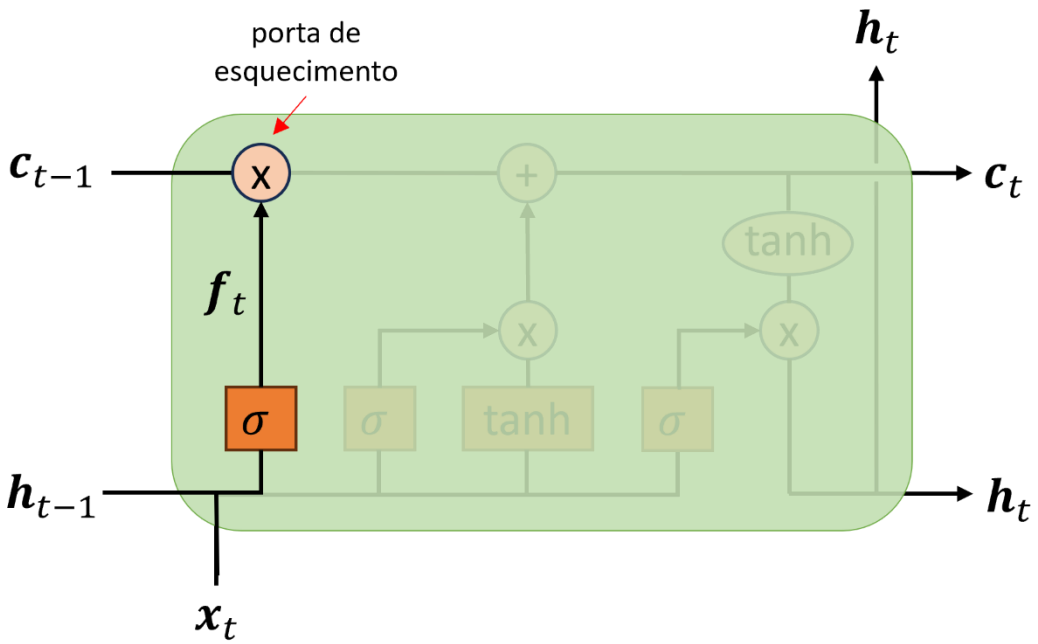
- Se $\boldsymbol{f}_t$ for sempre **1** e $\boldsymbol{i}_t$ for sempre **0**, então
$$\boldsymbol{c}_t = \boldsymbol{c}_{t-1}.$$
- Ou seja, o **estado interno da célula permanecerá constante para sempre**, passando inalterado para cada passo de tempo subsequente.
  - É assim que a memória de longo prazo é mantida.
- Assim, as **portas de entrada e de esquecimento** dão ao modelo a **capacidade de aprender quando (e o quanto) manter ou não** esse valor inalterado em resposta a entradas subsequentes.

Dinâmica das portas em uma célula LSTM



- Sempre que o_t for próximo de **1**, permitimos que o **estado interno atual da célula, c_t, impacte os instantes de tempo subsequentes** via h_t.
- Por outro lado, se o_t for próximo de **0**, impedimos que o **estado interno atual da célula, c_t, impacte os instantes de tempo subsequentes.**
- Assim
  - $o_t \rightarrow 1$: a **informação de c_t é importante** para o futuro.
  - $o_t \rightarrow 0$: a **informação de c_t não é relevante** para os próximos estados.

Porta de esquecimento

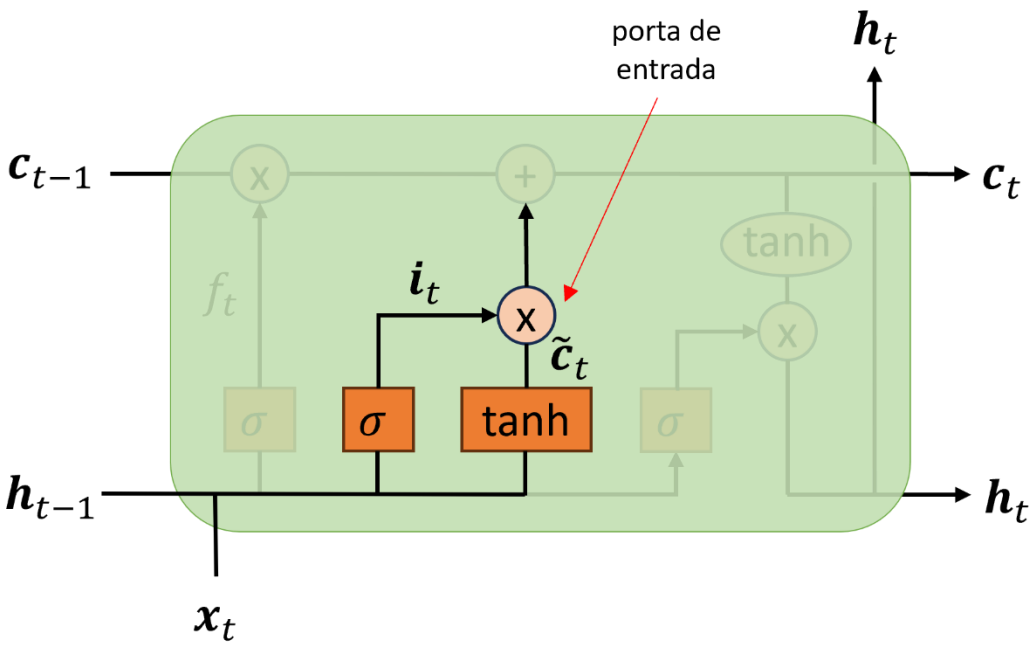


- **Controla** quais **informações** devem ser **descartadas** do estado de longo prazo

$$f_t = \begin{cases} \mathbf{0}, & \text{esquece completamente } \boldsymbol{c}_{t-1} \\ \mathbf{1}, & \text{mantém 100\% de } \boldsymbol{c}_{t-1} \end{cases}$$

- Porta controlada pela entrada atual $\boldsymbol{x}_t$ e pela saída anterior da célula, $\boldsymbol{h}_{t-1}$.
- Exemplo aplicado ao PLN:
  - O **estado de longo prazo** **mantém** o **gênero do sujeito** de uma frase para usar os pronomes corretos.
  - Ao ver um **novo sujeito**, a célula **esquece** o gênero do sujeito anterior.

Porta de entrada

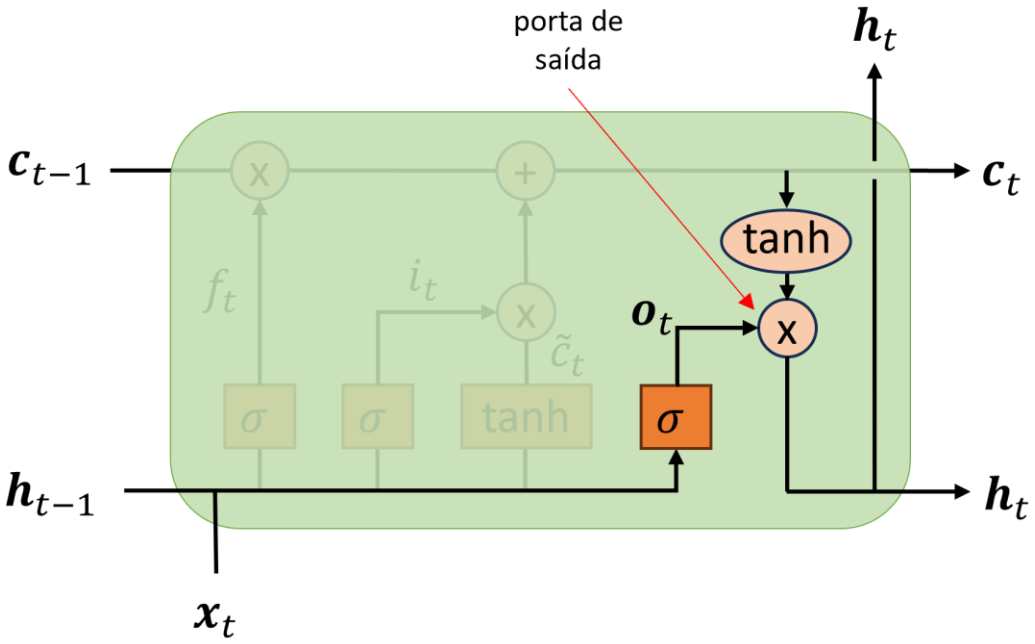


- **Controla o quanto atualizar o estado antigo**, c_{t-1}, com um novo candidato, $\tilde{c}_t$:

$$i_t = \begin{cases} 0, & \text{sem atualização} \\ 1, & \text{atualiza } c_{t-1} \text{ com } 100\% \text{ de } \tilde{c}_t \end{cases}$$

- Também controlada por x_t e h_{t-1}.
- No exemplo de PNL:
  - **Adiciona** o **gênero do novo sujeito** da frase ao **estado de longo prazo** da célula para **substituir** o antigo **que foi esquecido**.

Porta de saída



- A **saída da célula** é uma **versão filtrada** do **estado da célula**, $\boldsymbol{c_t}$, controlada por $\boldsymbol{x_t}$ e $\boldsymbol{h_{t-1}}$.
- $\boldsymbol{c_t}$ é passado através de uma função *tanh*, que comprime seus valores entre -1 e 1.
- Em seguida, a **porta decide o quanto de $\boldsymbol{c_t}$ será exposto na saída**

$$\boldsymbol{o_t} = \begin{cases} \mathbf{0}, & \text{sem saída} \\ \mathbf{1}, & \text{passa } 100\% \text{ do estado} \end{cases}$$

- No exemplo de PLN:
  - A célula **armazena** que o **sujeito é plural**. Quando um **verbo aparece**, a porta de saída **libera** essa **informação** para **escolher a conjugação correta e influenciar passos subsequentes**.

Resumo das equações de atualização da célula

- Vetor da porta de esquecimento

$$\boldsymbol{f}_t = \sigma\big([\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]\boldsymbol{W}_f + \boldsymbol{b}_f\big),$$

- Vetor da porta de entrada

$$\boldsymbol{i}_t = \sigma([\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]\boldsymbol{W}_i + \boldsymbol{b}_i),$$

- Vetor da porta de saída

$$\boldsymbol{o}_t = \sigma([\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]\boldsymbol{W}_o + \boldsymbol{b}_o),$$

- Vetor de informação candidata

$$\tilde{\boldsymbol{c}}_t = \tanh([\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]\boldsymbol{W}_c + \boldsymbol{b}_c)\,,$$

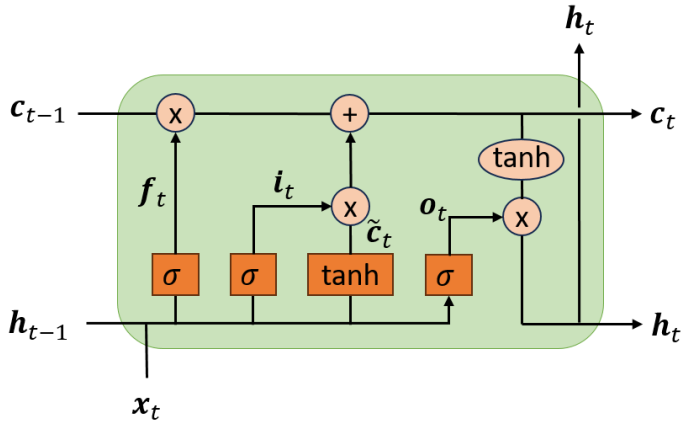
- Vetor de estado de longo prazo

$$\boldsymbol{c}_t = \underbrace{\boldsymbol{f}_t \boldsymbol{c}_{t-1}}_{\text{controla o esquecimento da informação anterior}} + \underbrace{\boldsymbol{i}_t \tilde{\boldsymbol{c}}_t}_{\text{controla a entrada de nova informação}},$$

- Vetor de estado de curto prazo

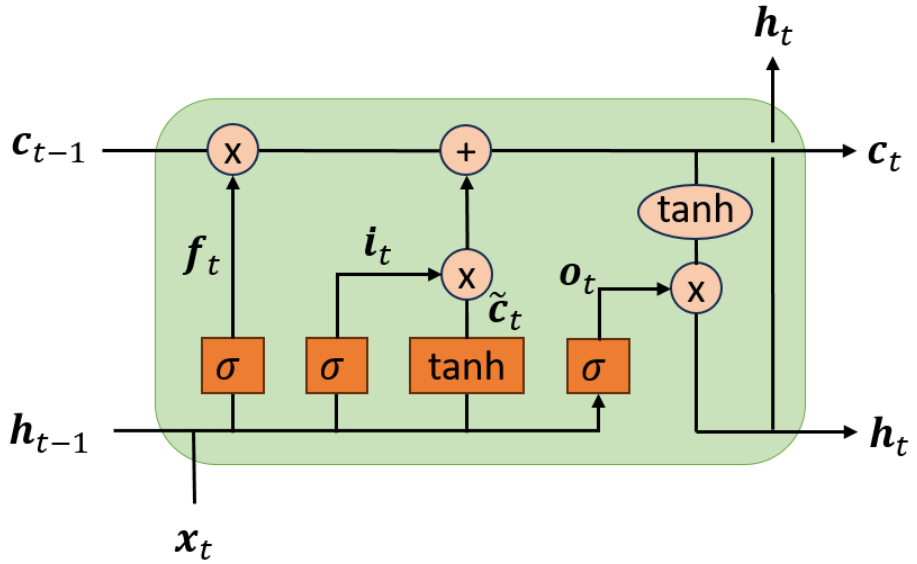
$$\boldsymbol{h}_t = \tanh(\boldsymbol{c}_t) \odot \boldsymbol{o}_t,$$

onde $\odot$ é o operador de multiplicação elemento a elemento.



Long-short term memory (LSTM)

- Resumindo, uma célula LSTM pode:
  - aprender a **reconhecer uma entrada importante e armazená-la** (totalmente ou parcialmente) no estado de longo prazo, $\boldsymbol{C}_t$,
    - Essas são as funções da **porta de entrada**
  - **preservá-la** pelo tempo que for necessário
    - Essa é a função da **porta de esquecimento**
  - e **expô-la** sempre que necessário
    - Essa é a função da **porta de saída**

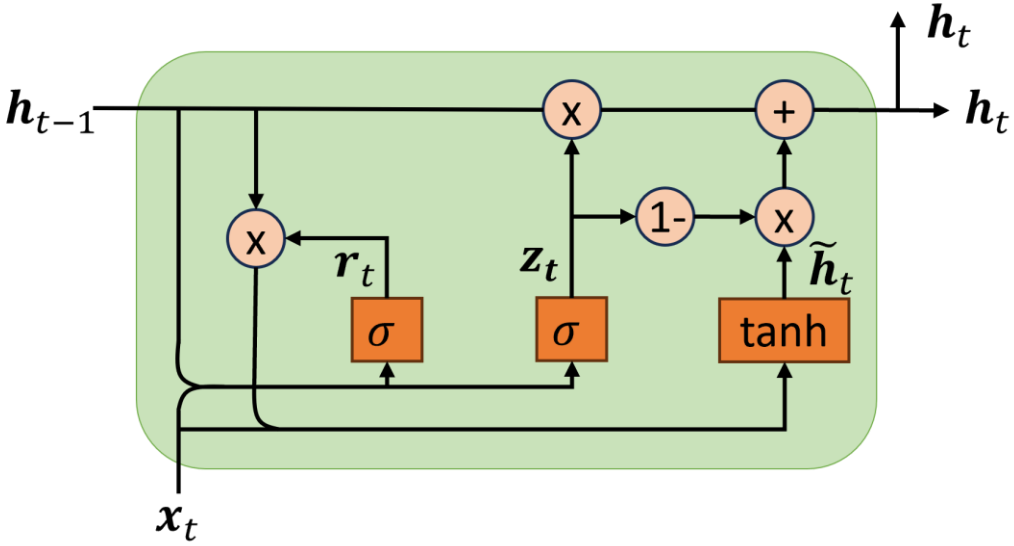


Principais desvantagens de LSTMs

- As saídas das portas **nunca são realmente 0 ou 1**.
  - Assim, o **conteúdo da célula inevitavelmente se corrompe após um longo período**.
  - Portanto, ainda apresenta **dificuldades com dependências temporais muito longas**.
- Alta **complexidade computacional**.
  - **Muito mais parâmetros** do que as RNNs tradicionais.
- **Dificuldade de paralelização**
  - **Processamento sequencial**: a célula depende de $t-1$ para calcular t.
  - Difícil explorar paralelismo em GPU.
- Hoje é **superada em muitos cenários**
  - Modelos mais modernos, como *Transformers* com *attention*, apresentam **melhor paralelização** e **capturam de dependências globais**.

Será que conseguimos algo quase tão bom quanto uma LSTM, porém mais simples?

Gated recurrent unit (GRU)

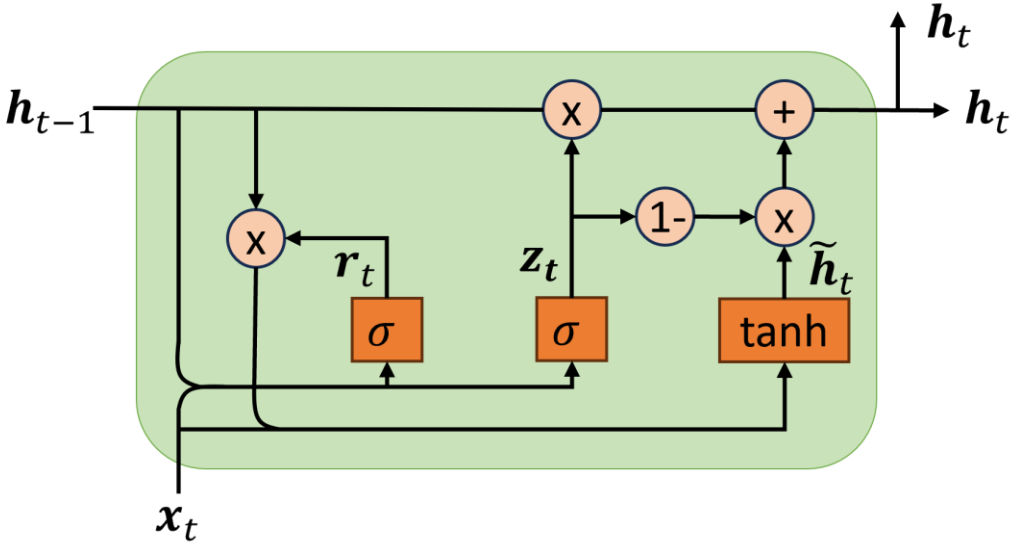


- Com o sucesso das LSTMs em capturar dependências de longo prazo, novas arquiteturas foram propostas com o objetivo de
  - reduzir a quantidade de parâmetros,
  - reduzir o custo computacional,
  - reduzir o consumo de memória,
  - e acelerar o treinamento

em relação às LSTMs.

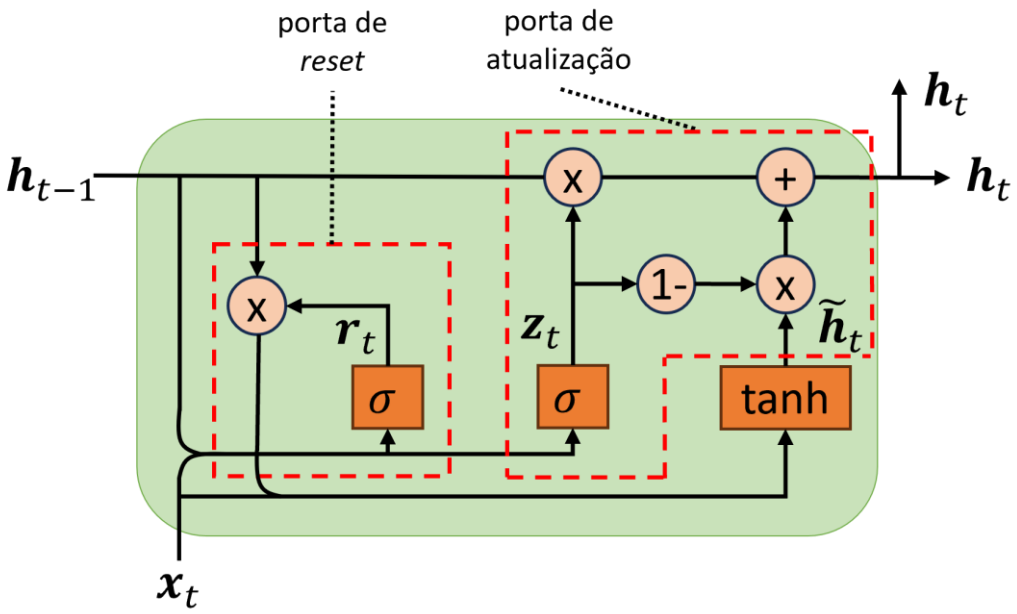
- Assim, as GRUs foram **propostas em 2014** por Kyunghyun Cho e colegas com esse objetivo.

Gated recurrent unit (GRU)



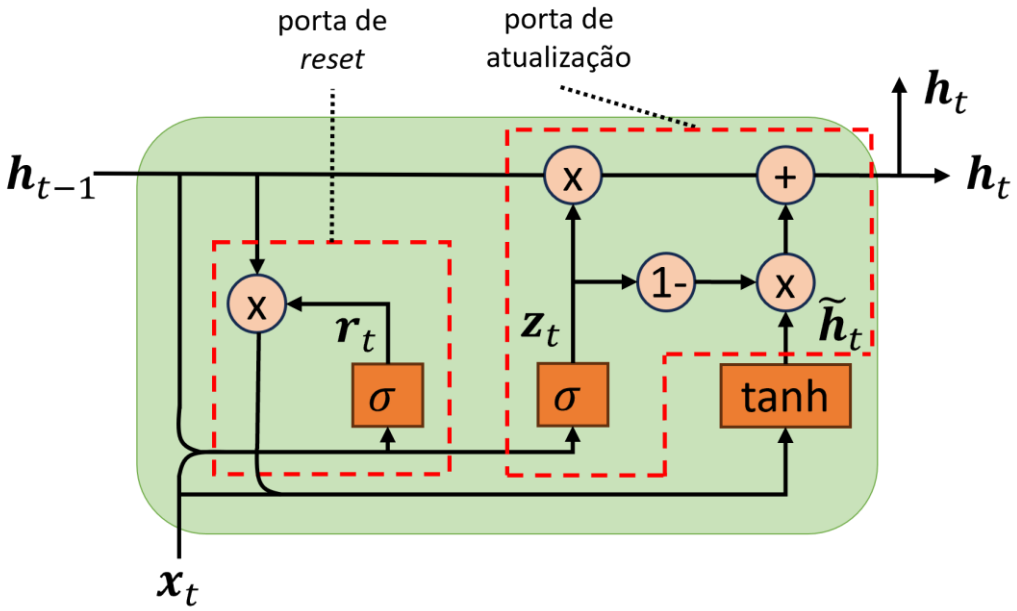
- As GRUs são uma **versão simplificada** da célula de memória LSTM.
- Em geral, **atingem desempenho comparável** às LSTMs, mas com a vantagem de terem **menos parâmetros**, serem **menos complexas** e serem **treinadas mais rapidamente**.
- Elas têm **apenas duas portas**: a de *reset* e a de atualização (controladas por $\boldsymbol{x}_t$ e $\boldsymbol{h}_{t-1}$).
- Além disso, elas **incorporam a memória de longo prazo no próprio estado oculto**.

Gated recurrent unit (GRU)



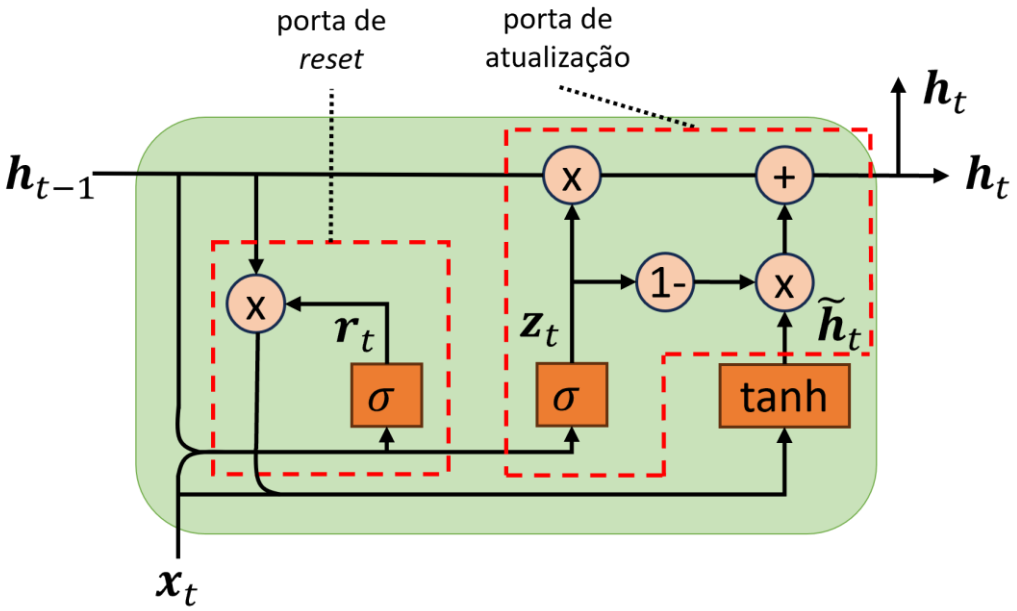
- A **porta de atualização** é responsável pela **memória de longo prazo**.
- Ela é a **combinação** das **portas de esquecimento** e de **entrada** da LSTM.
- Um **único vetor**, z_t, **controla** tanto a **porta de esquecimento** quanto a **porta de entrada**.

Gated recurrent unit (GRU)



- Se $z_t = 1$, a **porta de esquecimento** é **aberta** e a **porta de entrada** é **fechada** ($1 - z_t = 0$).
  - Retem-se o estado anterior, h_{t-1}.
- Se $z_t = 0$, a **porta de esquecimento** é **fechada** e a **porta de entrada** é **aberta** ($1 - z_t = 1$).
  - O estado oculto passa a ser $\tilde{h}_t$.
- Em outras palavras, sempre que uma **informação precisa ser armazenada**, o **local** (i.e., elemento) **onde ela será armazenada é apagado primeiro**.

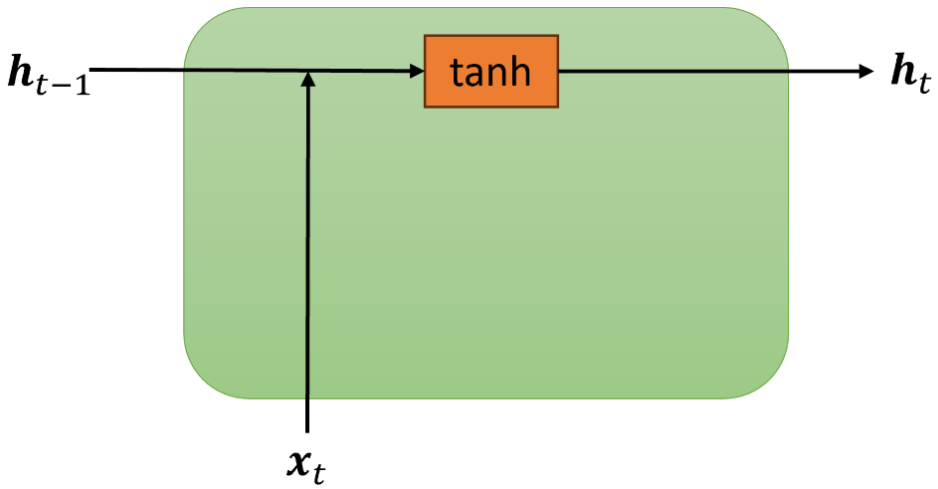
Gated recurrent unit (GRU)



- A porta de *reset* é responsável pela **memória de curto prazo**.
- Ela decide **quanta informação passada**, $\boldsymbol{h_{t-1}}$, será **mantida ou descartada**.
- A célula GRU **não tem uma porta de saída**.
- **O vetor de estado oculto**, $\boldsymbol{h_t}$, **completo é apresentado na saída** da célula a cada passo de tempo.

Como as células LSTM e GRU minimizam o problema do desaparecimento do gradiente e, consequentemente, mantém a memória de longo prazo?

Desaparecimento do gradiente

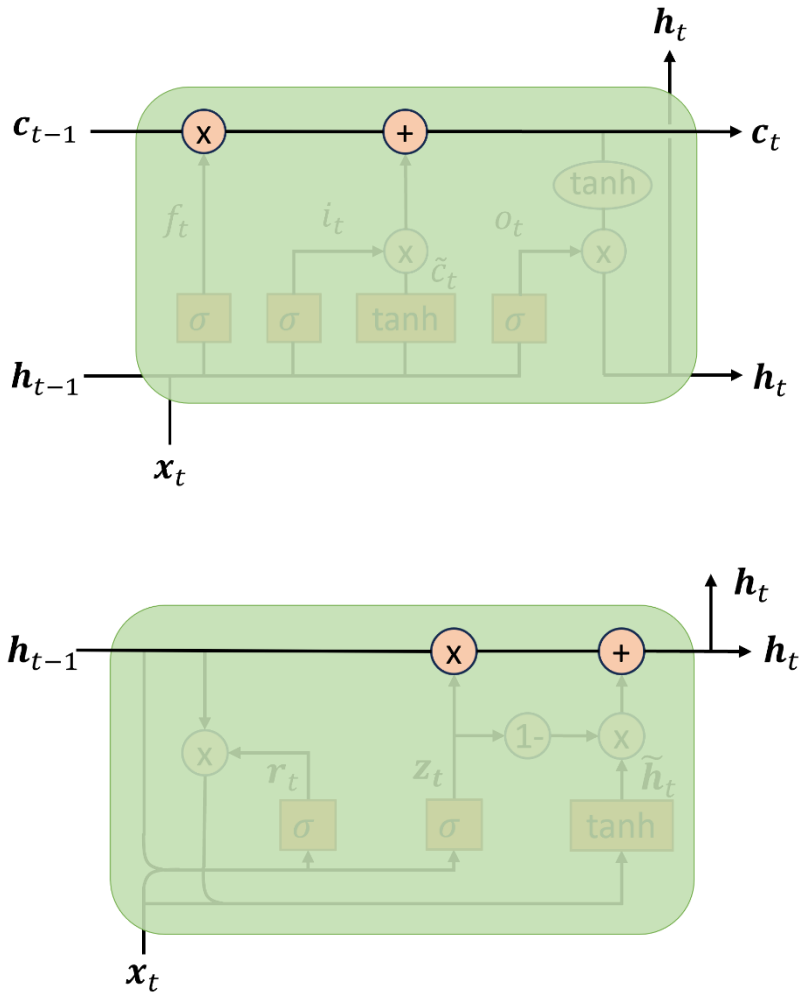


- As células de RNNs tradicionais apresentam um **caminho não-linear para a retropropagação do gradiente**.
- O **novo estado da célula passa por uma função tanh**

$$\boldsymbol{h}_t = \tanh(\boldsymbol{W}[\boldsymbol{x}_t, \boldsymbol{h}_{t-1}]) .$$

- Como sabemos, a derivada da função tanh tem um valor máximo igual 1.
- Como vimos, o resultado é que o **gradiente diminui exponencialmente** à medida que volta no tempo, **impedindo que a rede aprenda dependências de longo prazo**.

Desaparecimento do gradiente



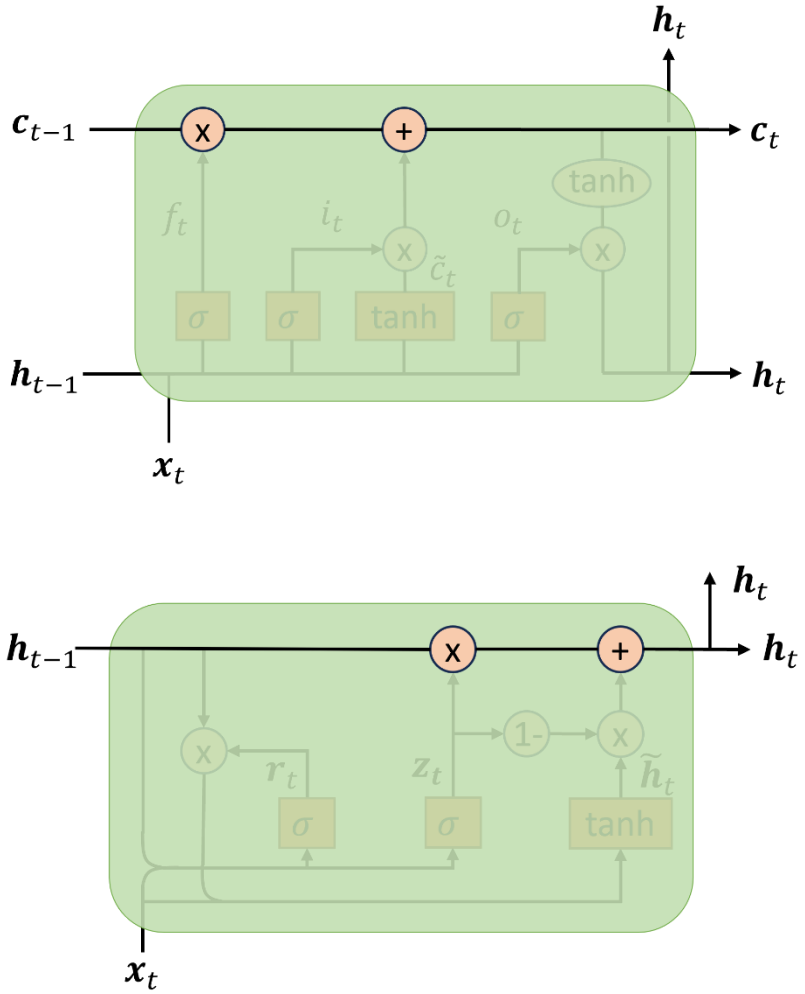
- Já as células de LSTMs e GRUs **modificam o estado da célula, $\boldsymbol{c}_t$, por meio de um caminho controlável** (por $\boldsymbol{f}_t$, $\boldsymbol{i}_t$ e $\boldsymbol{z}_t$) que **contém apenas interações lineares**

$$\boldsymbol{c}_t = \boldsymbol{f}_t \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \tilde{\boldsymbol{c}}_t \text{ (LSTM)}$$

$$\boldsymbol{h}_t = \boldsymbol{z}_t \boldsymbol{h}_{t-1} + (\boldsymbol{1} - \boldsymbol{z}_t) \tilde{\boldsymbol{h}}_t \text{ (GRU)}$$

- $\boldsymbol{c}_t$ e $\boldsymbol{h}_t$ são atualizados usando-se **uma soma e multiplicações elemento a elemento**, **sem passar por transformações não lineares**.

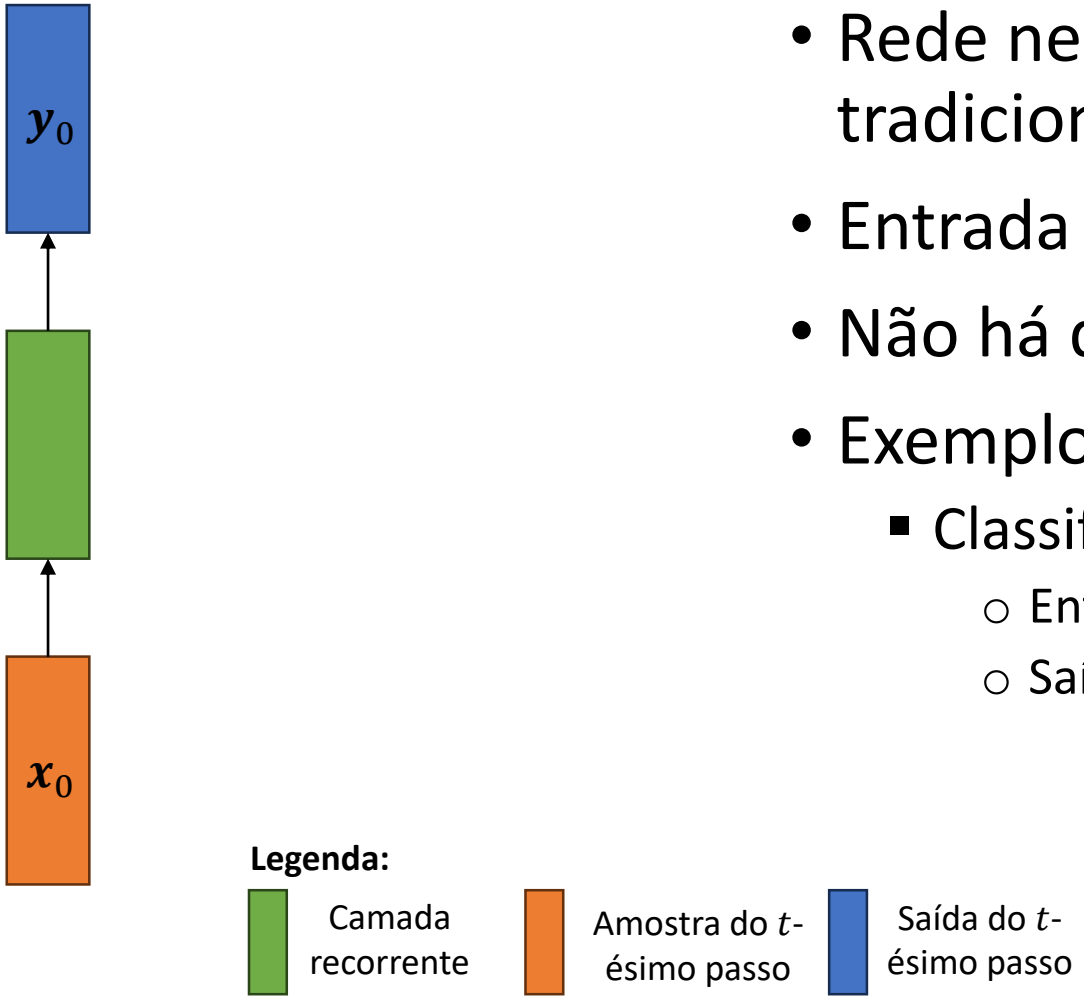
Desaparecimento do gradiente



- Esse **caminho controlável** **permite que os gradientes retrocedam por longos instantes de tempo** sem **desaparecer**.
- A rede **aprende** a decidir quando deixar o **gradiente desaparecer** e quando **preservá-lo**.
- Portanto, as portas permitem que a rede decida o que guardar e o que descartar.

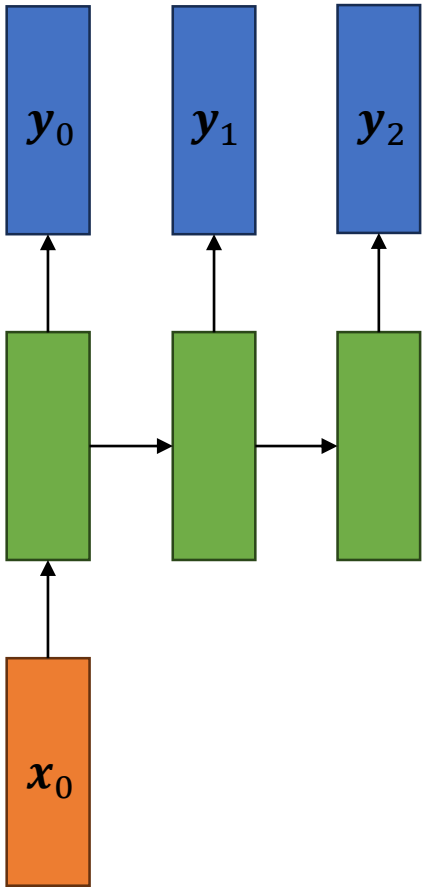
Diferentes aplicações para as RNNs

Aplicação: um-para-um



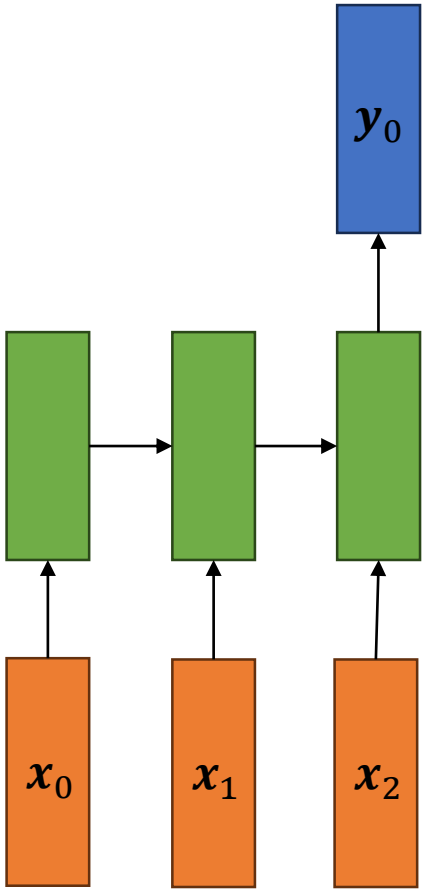
- Rede neural de alimentação direta tradicional.
- Entrada e saída de tamanhos fixos.
- Não há dados sequenciais envolvidos.
- Exemplo de uso:
  - Classificação de imagens.
    - Entrada: imagem
    - Saída: classe

Aplicação: um-para-vários



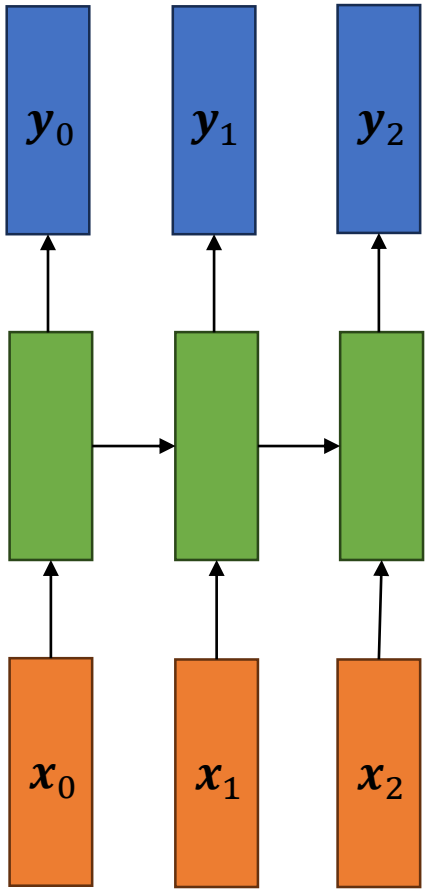
- Entrada de tamanho fixo.
- Saída de tamanho variável (sequência).
- Rede do tipo vetor para sequência
- Exemplo de uso:
  - Legendas de imagens.
    - Entrada: uma imagem
    - Saída: sequência de palavras descrevendo a imagem
  - Geração de música.
    - Entrada: um vetor que representa um gênero musical
    - Saída: as amostras de áudio da música

Aplicação: vários-para-um



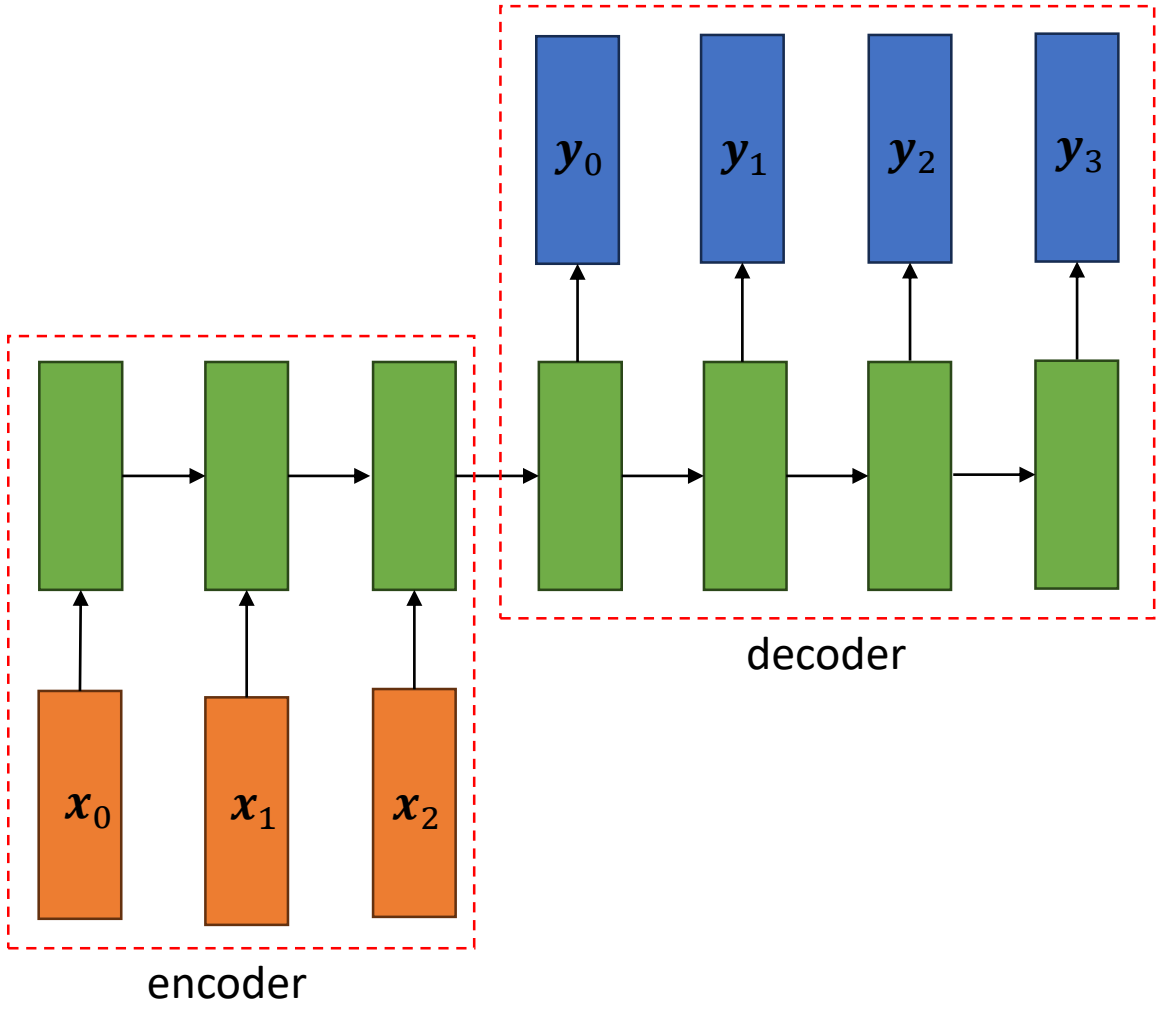
- Entrada de tamanho variável (sequência).
- Saída de tamanho fixo.
- Rede do tipo sequência para vetor.
- Exemplo de uso:
  - Classificação de sentimentos: por exemplo, classificar o sentimento de uma crítica de filme:
    - Entrada: sequência de palavras correspondentes à crítica
    - Saída: pontuação da crítica: 0 [odiou] até 1 [amou]

Aplicação: vários-para-vários síncrono



- Entrada de tamanho variável (sequência).
- Saída de tamanho variável (sequência).
- Rede do tipo sequência para sequência
- Entrada e saída sincronizadas.
  - Comprimentos de entrada e saída são iguais.
- Exemplo de uso:
  - Classificação de vídeo quadro a quadro.
    - Monitoramento de tráfego:
      - ✓ Entrada: sequência de imagens da vista de um semáforo.
      - ✓ Saída: quantidade de carros em cada imagem.

Aplicação: vários-para-vários assíncrono



- Entrada de tamanho variável (sequência).
- Saída de tamanho variável (sequência).
- Rede do tipo *encoder/decoder*.
- Entrada e saída não são sincronizadas.
  - Comprimentos de entrada e saída não são necessariamente iguais.
- Exemplo de uso:
  - Tradução automática
    - Entrada: sequência de palavras em uma língua.
    - Saída: sequência de palavras em outra língua.
  - *chatbots*

Exemplo

- Exemplo: rnn.ipynb



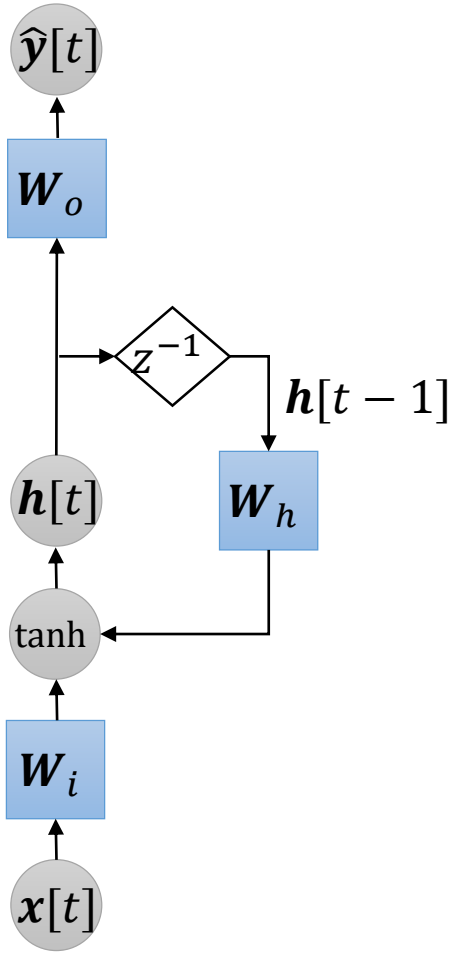
Lista de exercícios

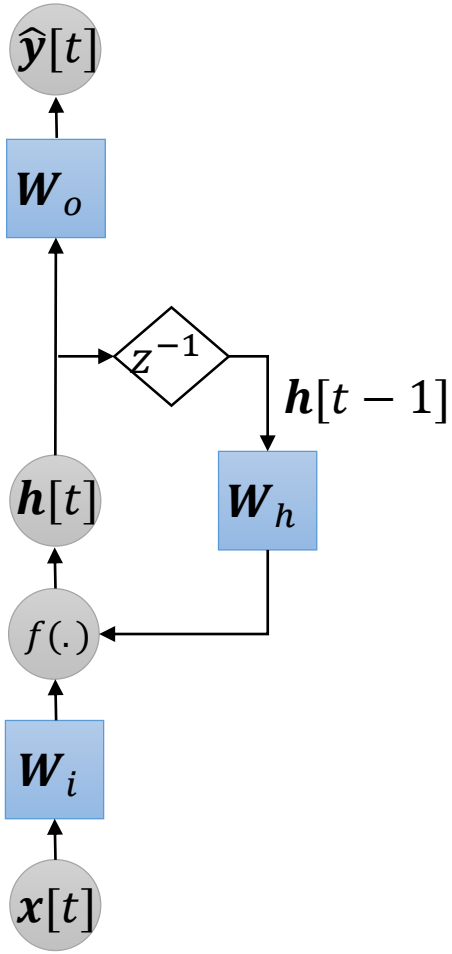
Lista #11 - RNN

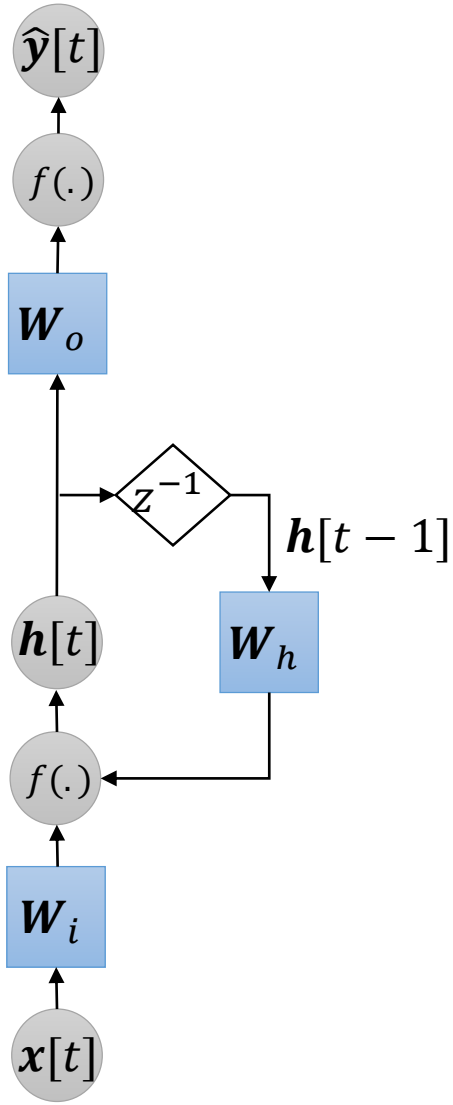
Perguntas?

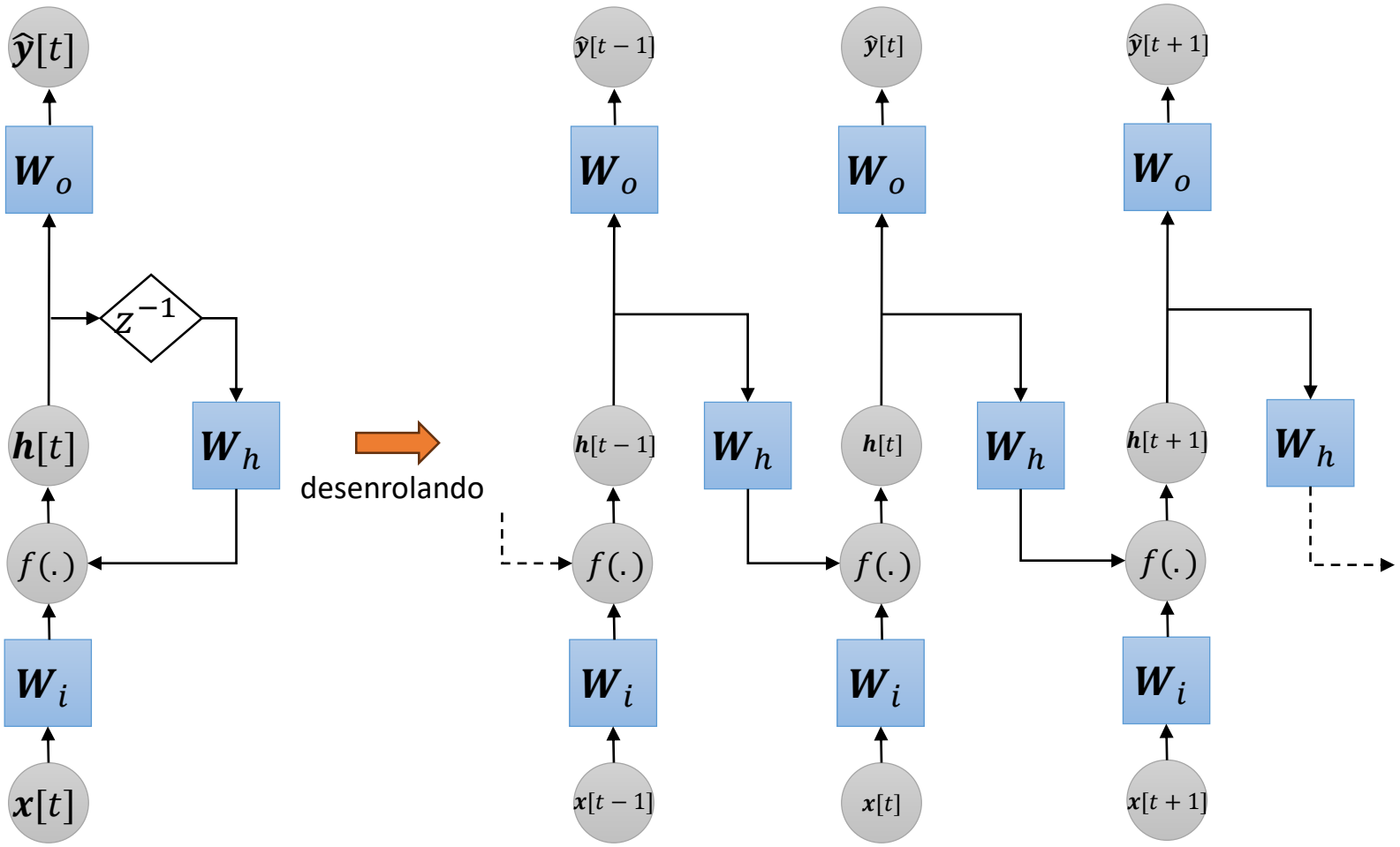
Obrigado!

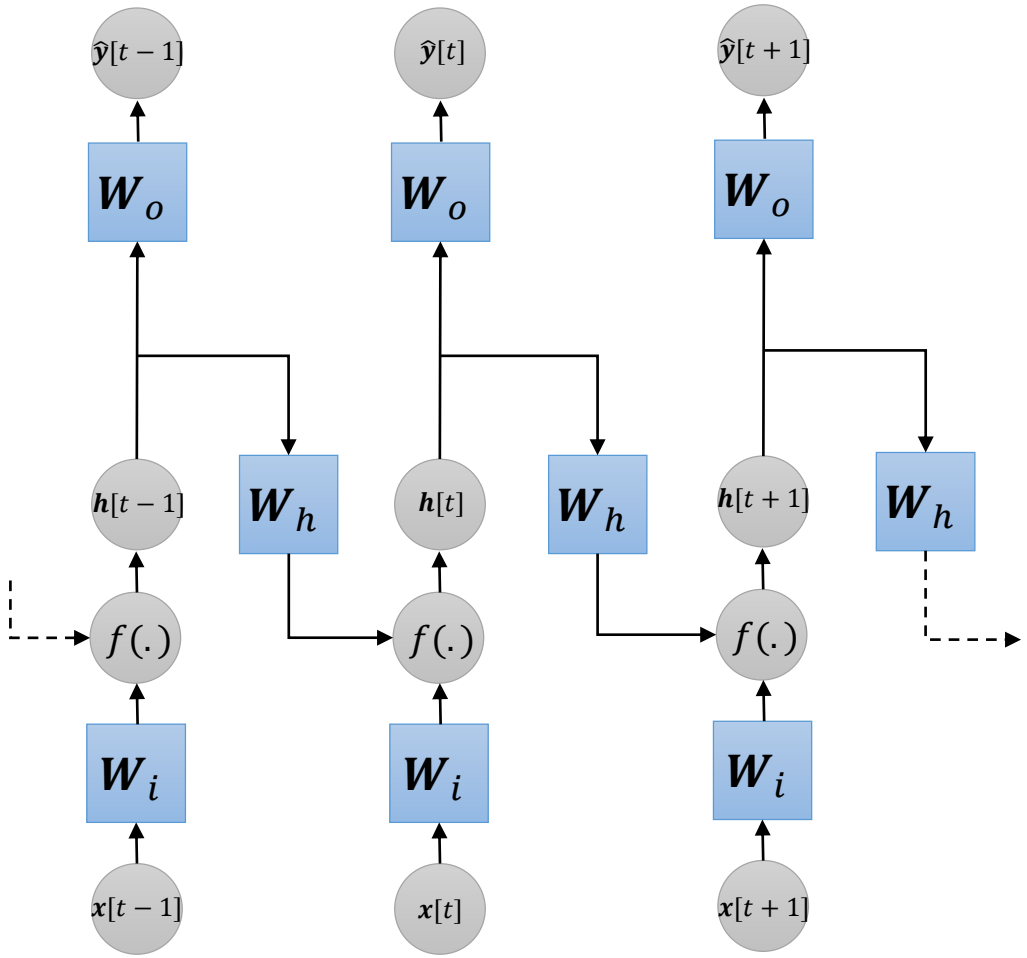
Figuras

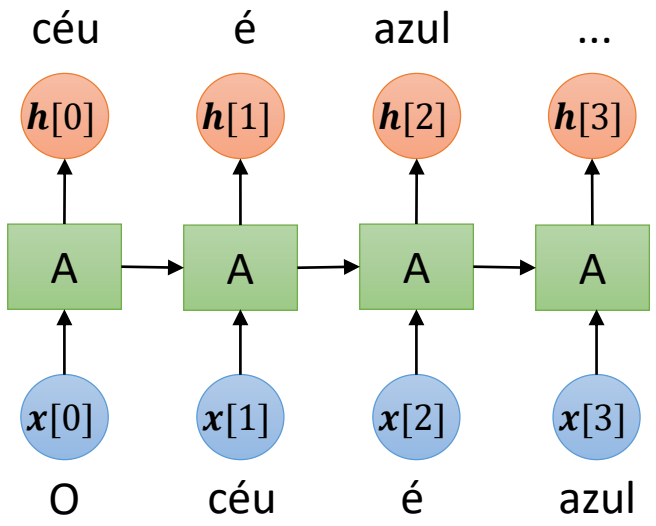

$\hat{\boldsymbol{y}}[t]$
$\boldsymbol{W}_o$
z^{-1}
$\boldsymbol{h}[t-1]$
$\boldsymbol{h}[t]$
$\boldsymbol{W}_h$
tanh
$\boldsymbol{W}_i$
$\boldsymbol{x}[t]$

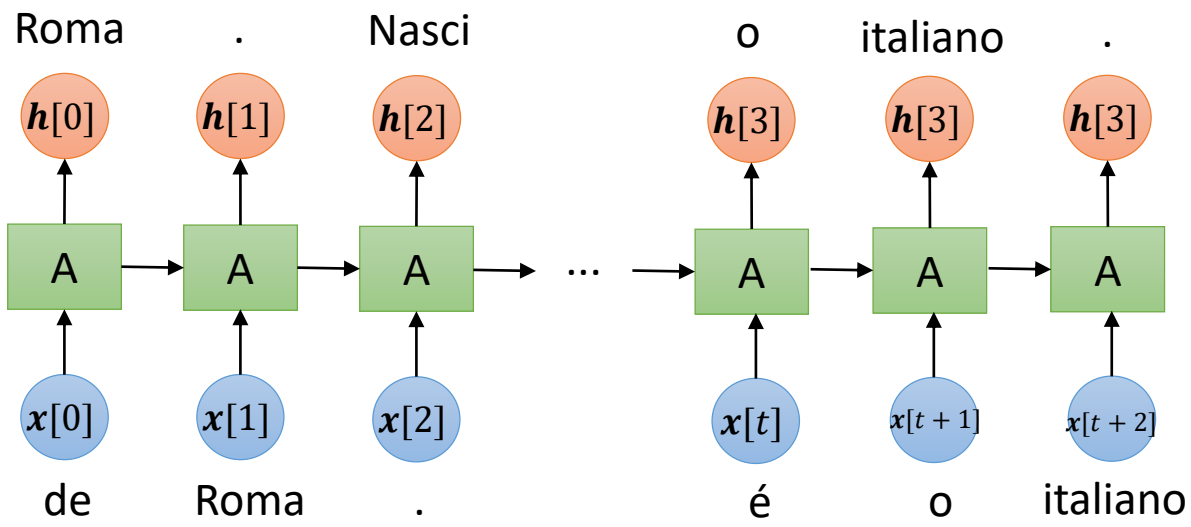

$\hat{\boldsymbol{y}}[t]$
$\boldsymbol{W}_o$
z^{-1}
$\boldsymbol{h}[t-1]$
$\boldsymbol{h}[t]$
$\boldsymbol{W}_h$
$f(.)$
$\boldsymbol{W}_i$
$\boldsymbol{x}[t]$

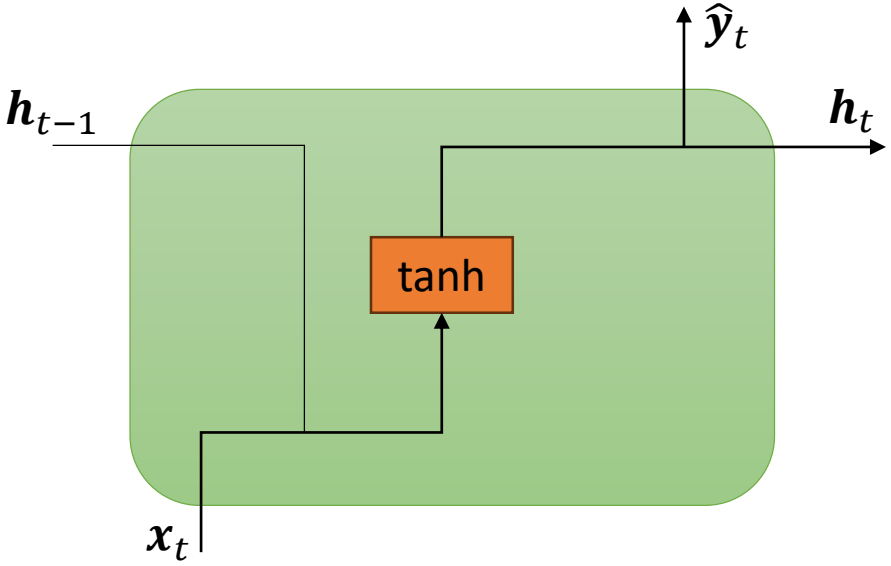

$\hat{\boldsymbol{y}}[t]$
$f(.)$
$\boldsymbol{W}_o$
z^{-1}
$\boldsymbol{h}[t-1]$
$\boldsymbol{h}[t]$
$\boldsymbol{W}_h$
$f(.)$
$\boldsymbol{W}_i$
$\boldsymbol{x}[t]$

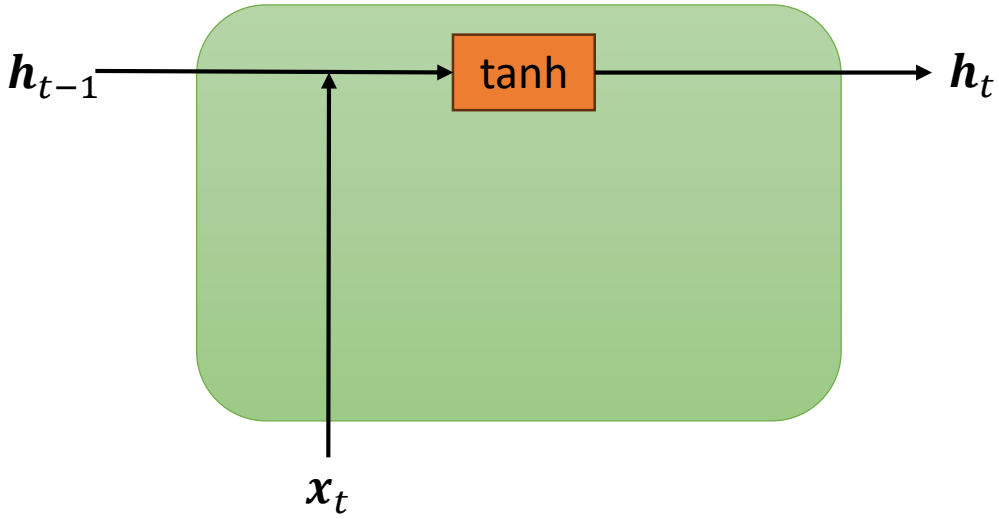

$\hat{y}[t]$
W_o
z^{-1}
$h[t]$
W_h
$f(.)$
W_i
$x[t]$
desenrolando
$\hat{y}[t-1]$
$\hat{y}[t]$
$\hat{y}[t+1]$
$h[t-1]$
$h[t+1]$
$x[t-1]$
$x[t+1]$

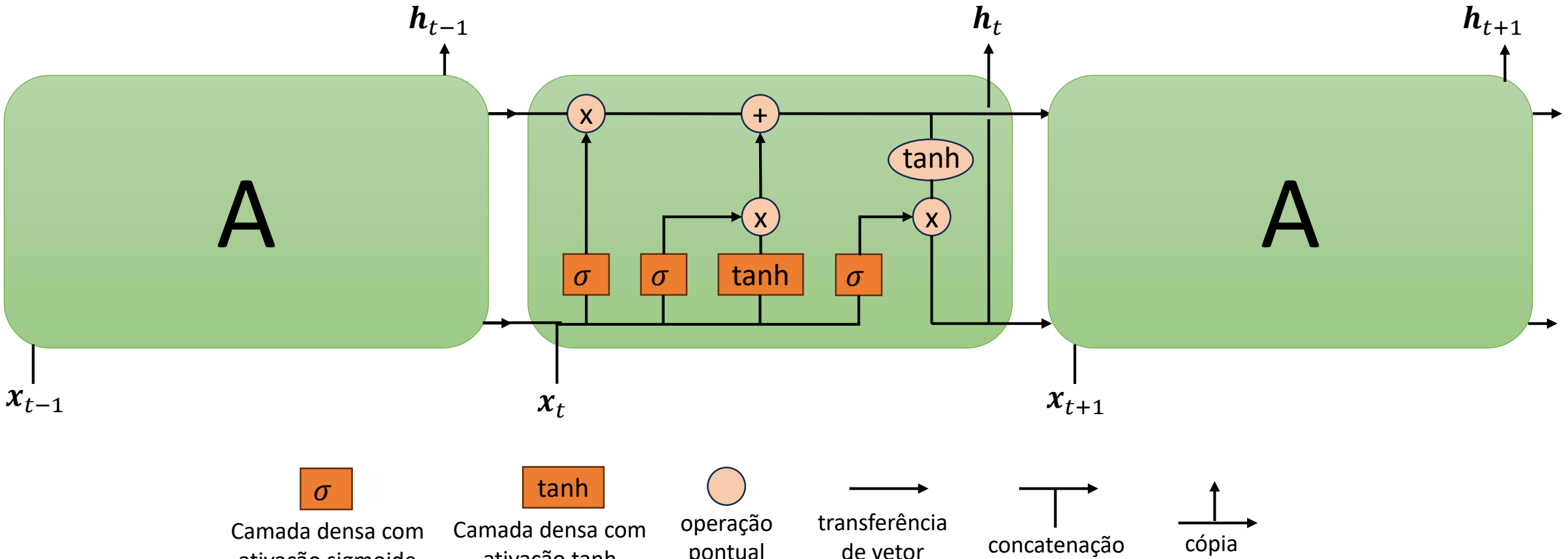

$\hat{y}[t-1]$
$\hat{y}[t]$
$\hat{y}[t+1]$
W_o
W_o
W_o
$h[t-1]$
W_h
$h[t]$
W_h
$h[t+1]$
W_h
$f(.)$
$f(.)$
$f(.)$
W_i
W_i
W_i
$x[t-1]$
$x[t]$
$x[t+1]$

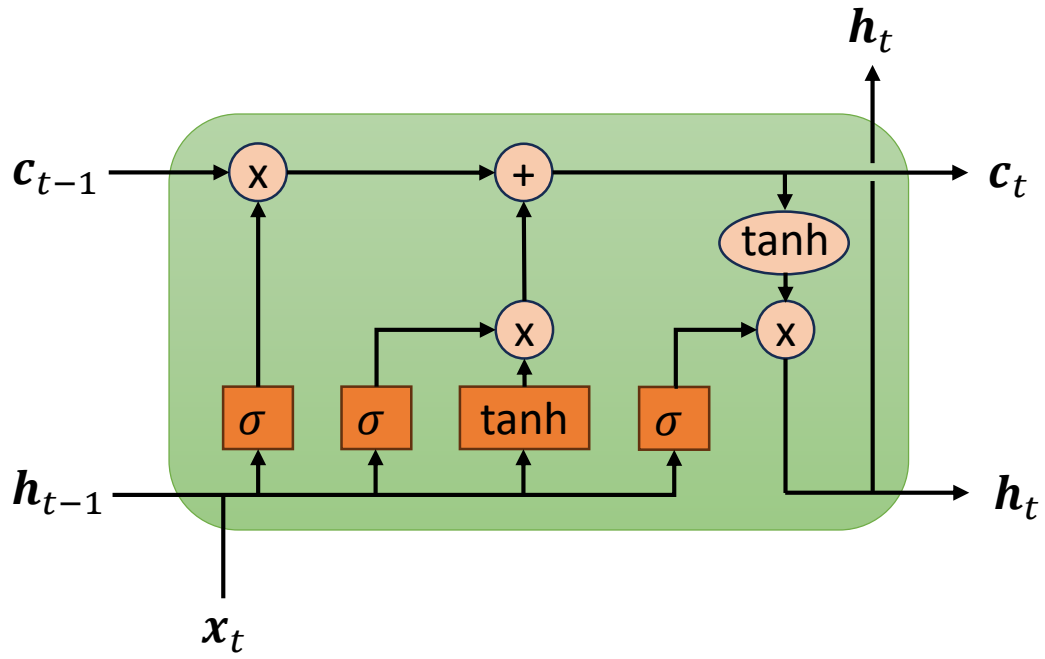

céu
é
azul
...
$\boldsymbol{h}[0]$
$\boldsymbol{h}[1]$
$\boldsymbol{h}[2]$
$\boldsymbol{h}[3]$
A
A
A
A
$\boldsymbol{x}[0]$
$\boldsymbol{x}[1]$
$\boldsymbol{x}[2]$
$\boldsymbol{x}[3]$
O
céu
é
azul


Roma
.
Nasci
o
italiano
.
h[0]
h[1]
h[2]
h[3]
h[3]
h[3]
A
A
A
...
A
A
A
x[0]
x[1]
x[2]
x[t]
x[t + 1]
x[t + 2]
de
Roma
.
é
o
italiano


$\hat{y}_t$
h_{t-1}
h_t
tanh
x_t


h_{t-1}
tanh
h_t
x_t


h_{t-1}
h_t
h_{t+1}
A
A
x
+
tanh
x
x
σ
σ
tanh
σ
x_{t-1}
x_t
x_{t+1}
σ
Camada densa com ativação sigmoide
tanh
Camada densa com ativação tanh
operação pontual
transferência de vetor
concatenação
cópia



σ
Camada totalmente conectada com ativação sigmoide

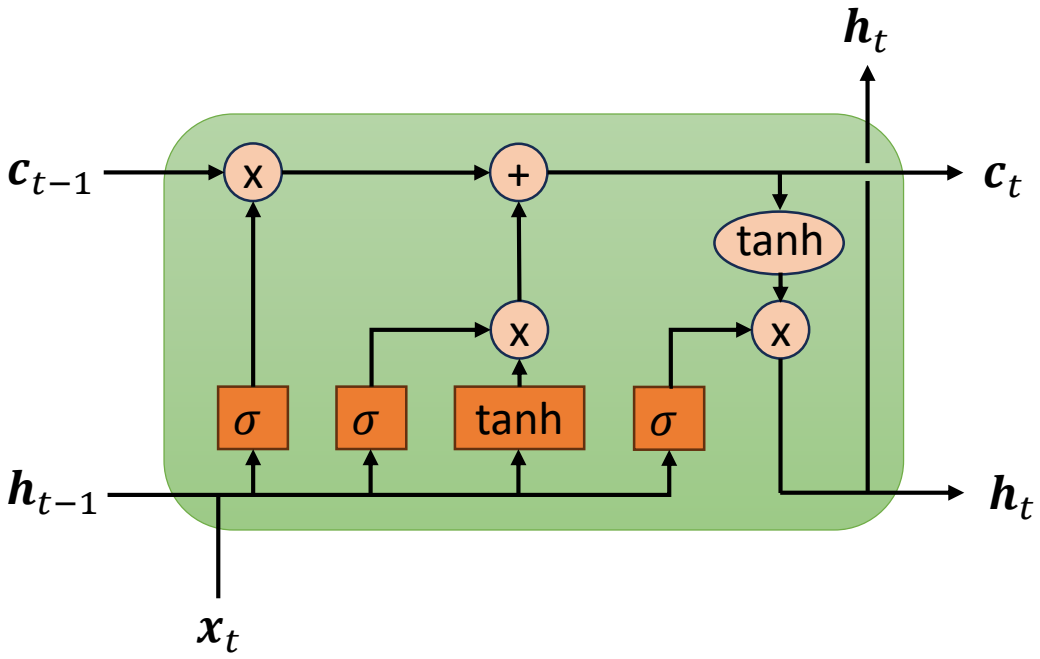
tanh
Camada totalmente conectada com ativação tanh

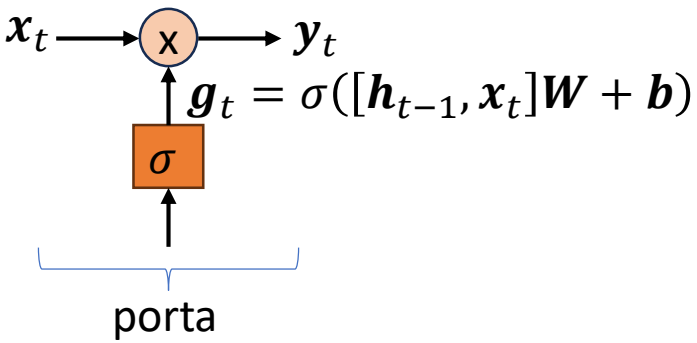
operação elemento a elemento

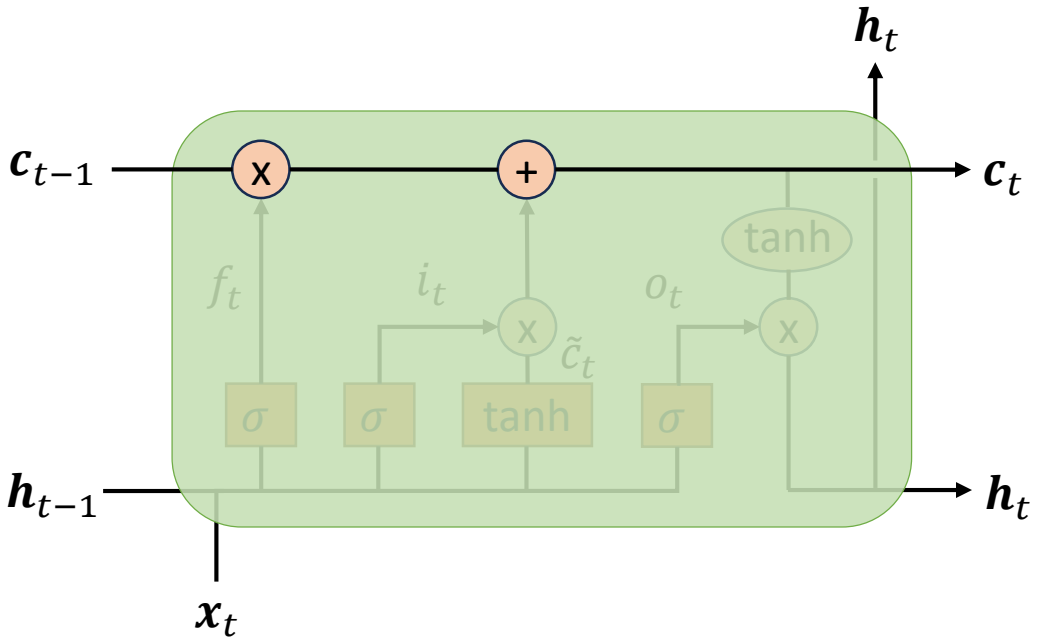
transferência de vetor

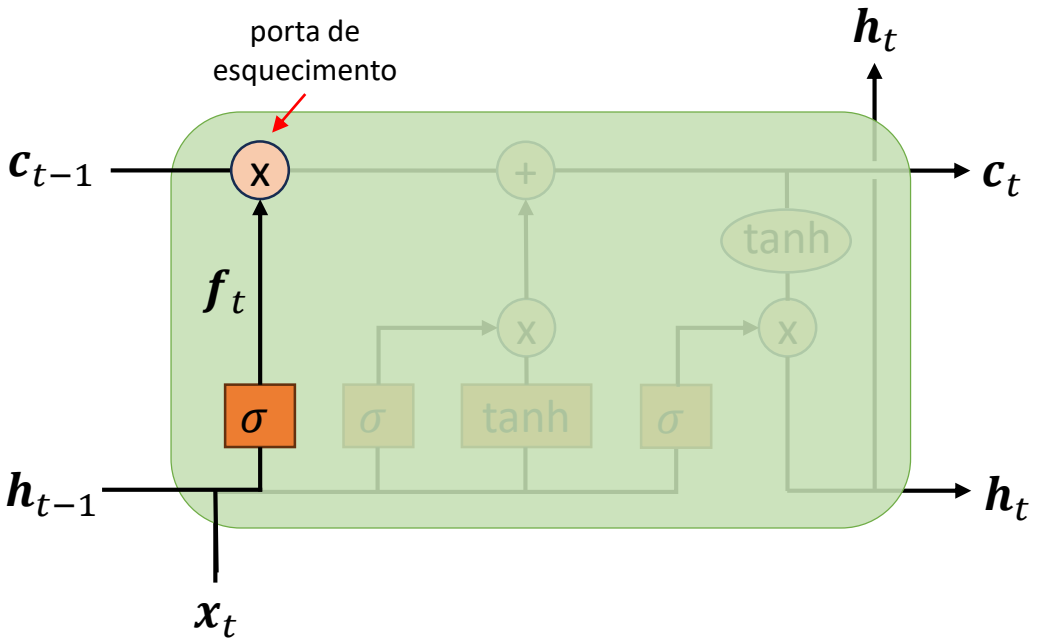
concatenação de vetores

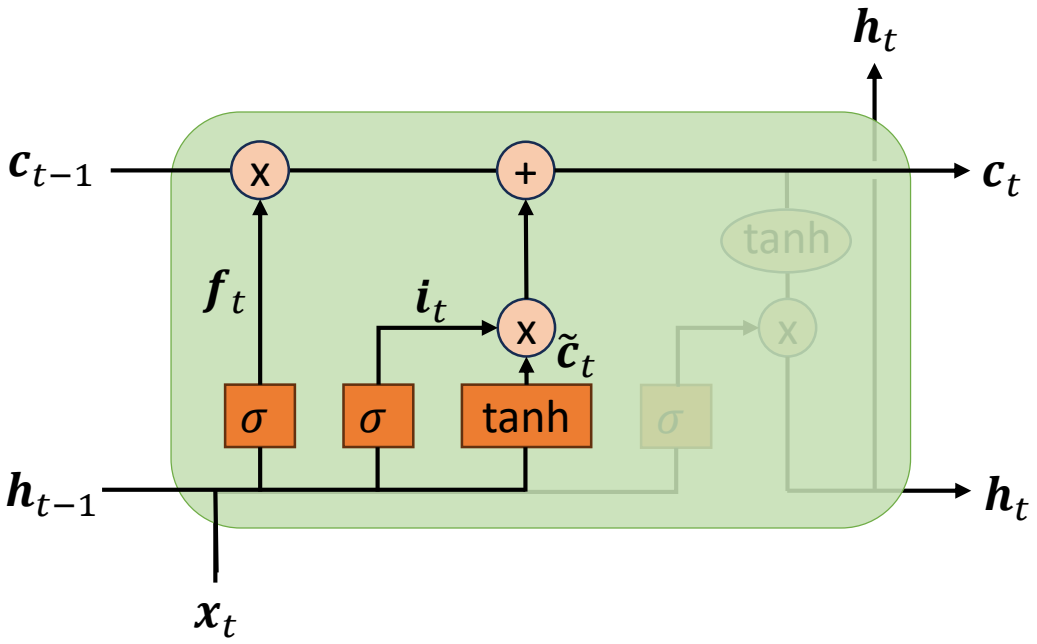
cópia de vetor

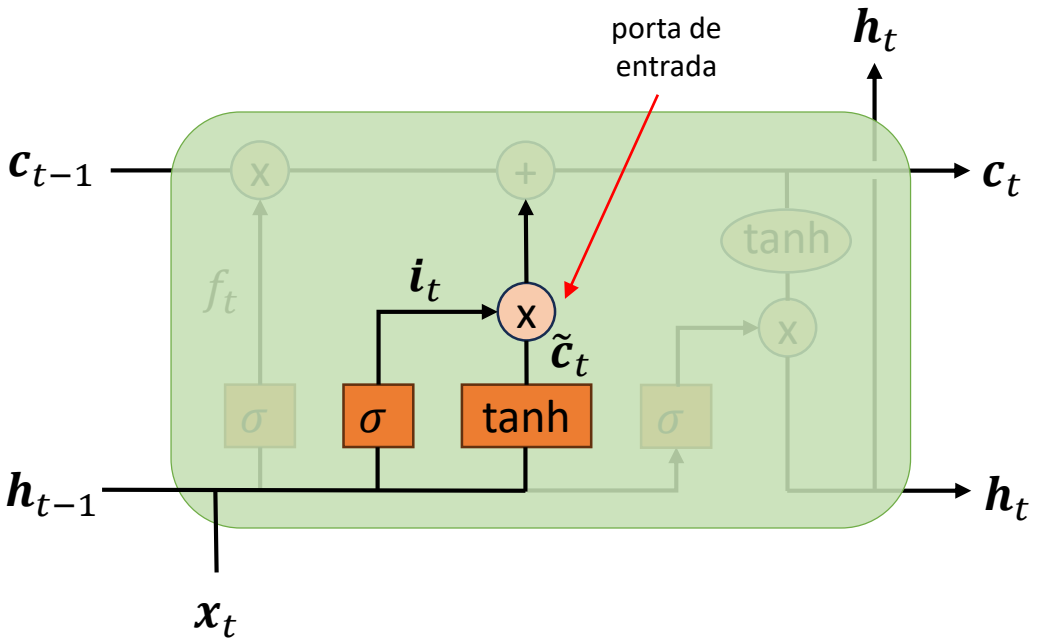

h_t
c_{t-1}
x
+
c_t
tanh
x
x
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

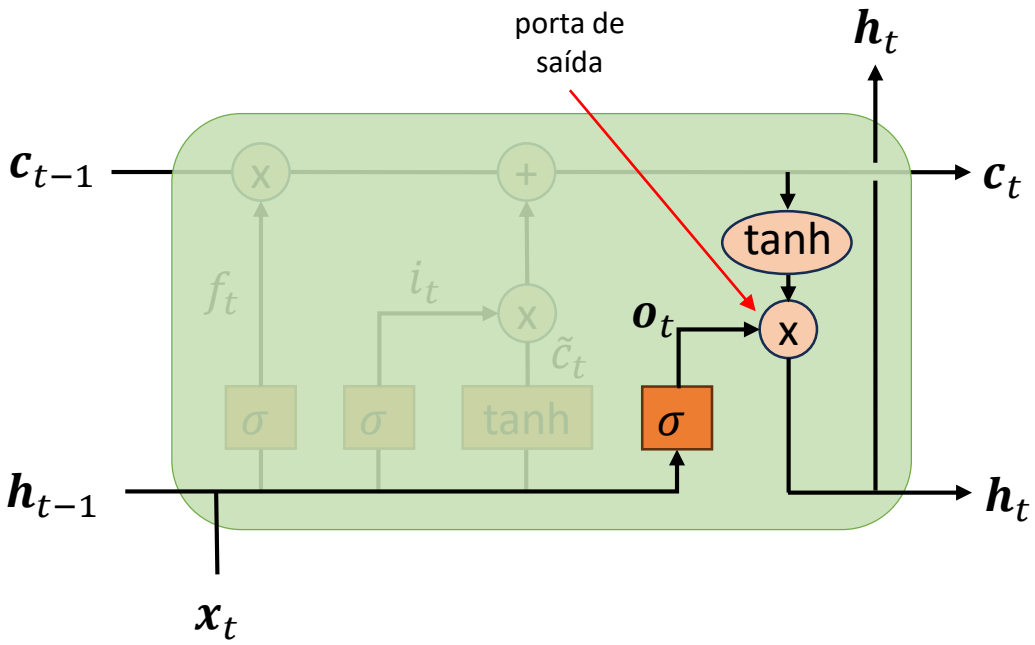

x_t
x
y_t
$g_t = \sigma([h_{t-1}, x_t]W + b)$
σ
porta

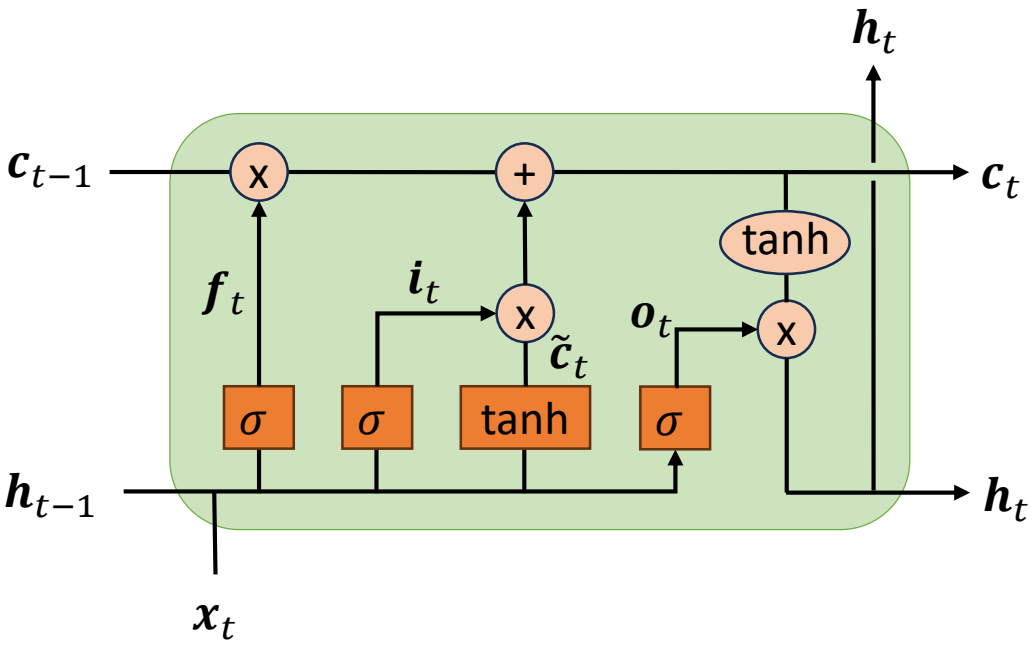

h_t
c_{t-1}
x
+
c_t
tanh
f_t
i_t
o_t
x
x
$\tilde{c}_t$
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

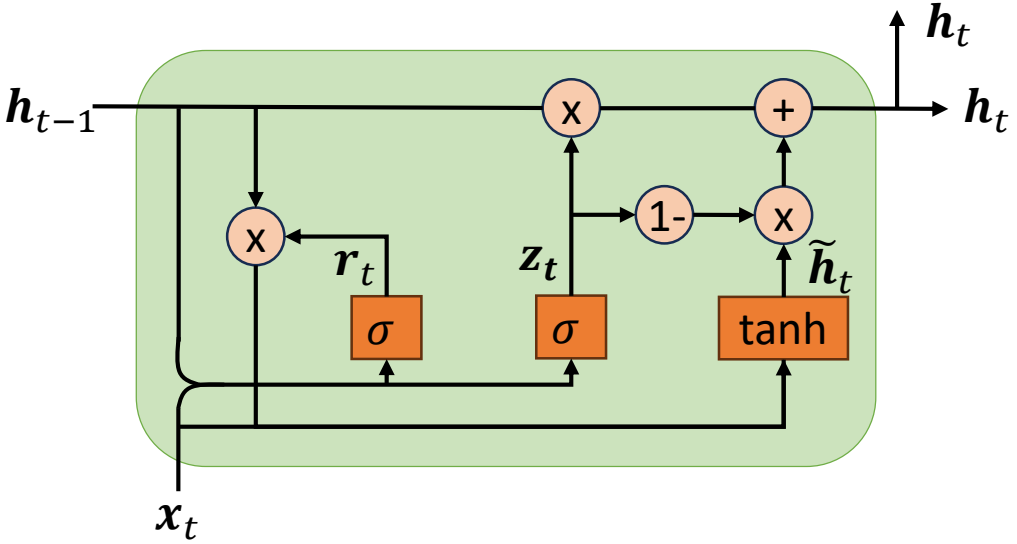

porta de
esquecimento
h_t
c_{t-1}
x
+
c_t
tanh
f_t
x
x
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

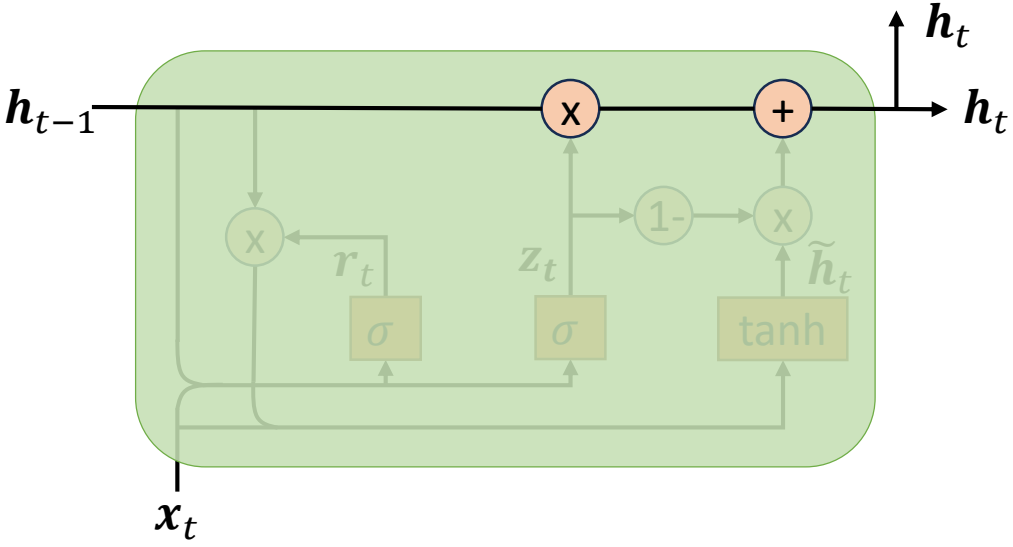

h_t
c_{t-1}
x
+
c_t
tanh
f_t
i_t
x
$\tilde{c}_t$
x
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

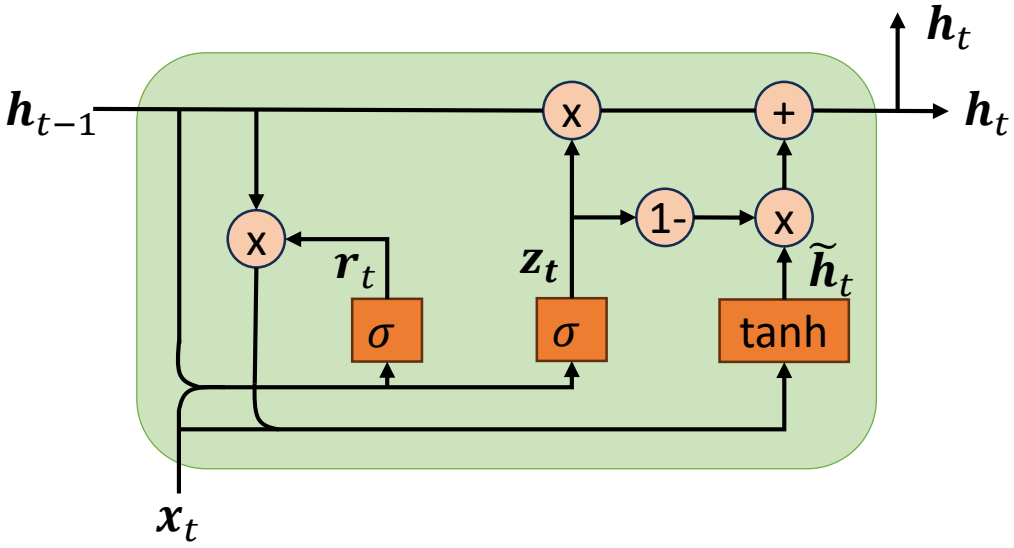

porta de
entrada
h_t
c_{t-1}
c_t
x
+
tanh
f_t
i_t
x
$\tilde{c}_t$
σ
σ
tanh
σ
x
h_{t-1}
h_t
x_t

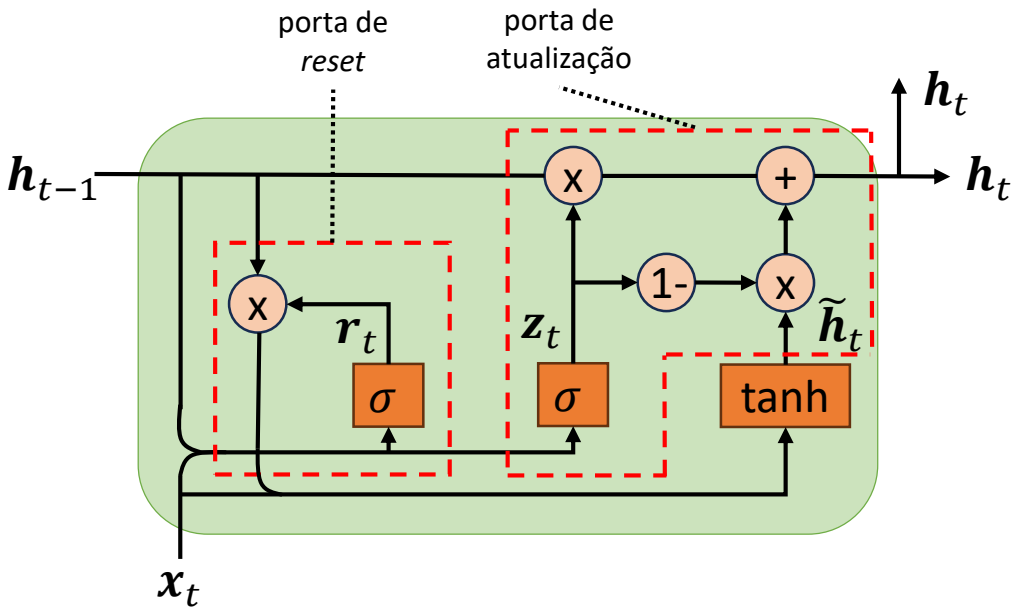

porta de
saída
h_t
c_{t-1}
c_t
x
+
tanh
f_t
i_t
x
o_t
x
$\tilde{c}_t$
σ
σ
tanh
σ
h_{t-1}
h_t
x_t


h_t
c_{t-1}
x
+
c_t
tanh
f_t
i_t
x
o_t
x
$\tilde{c}_t$
σ
σ
tanh
σ
h_{t-1}
h_t
x_t

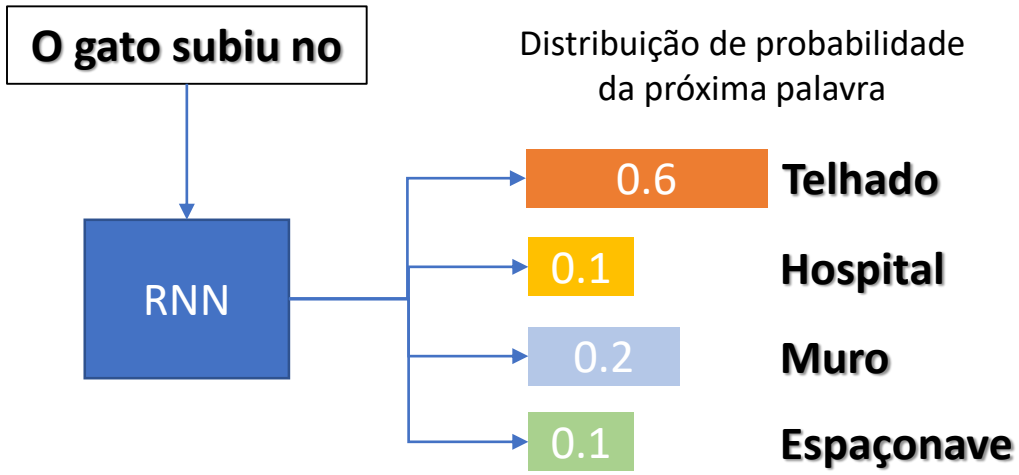

h_t
h_{t-1}
h_t
x
+
x
r_t
1-
z_t
x
$\tilde{h}_t$
σ
σ
tanh
x_t


h_t
h_{t-1}
h_t
x
+
x
r_t
1-
z_t
x
$\tilde{h}_t$
σ
σ
tanh
x_t


h_t
h_{t-1}
h_t
x
+
x
r_t
1-
z_t
x
$\tilde{h}_t$
σ
σ
tanh
x_t


porta de
reset
porta de
atualização
h_t
h_{t-1}
x
+
h_t
x
1-
x
r_t
z_t
$\tilde{h}_t$
σ
σ
tanh
x_t

Palavras anteriores (contexto)



A palavra com maior
probabilidade é escolhida